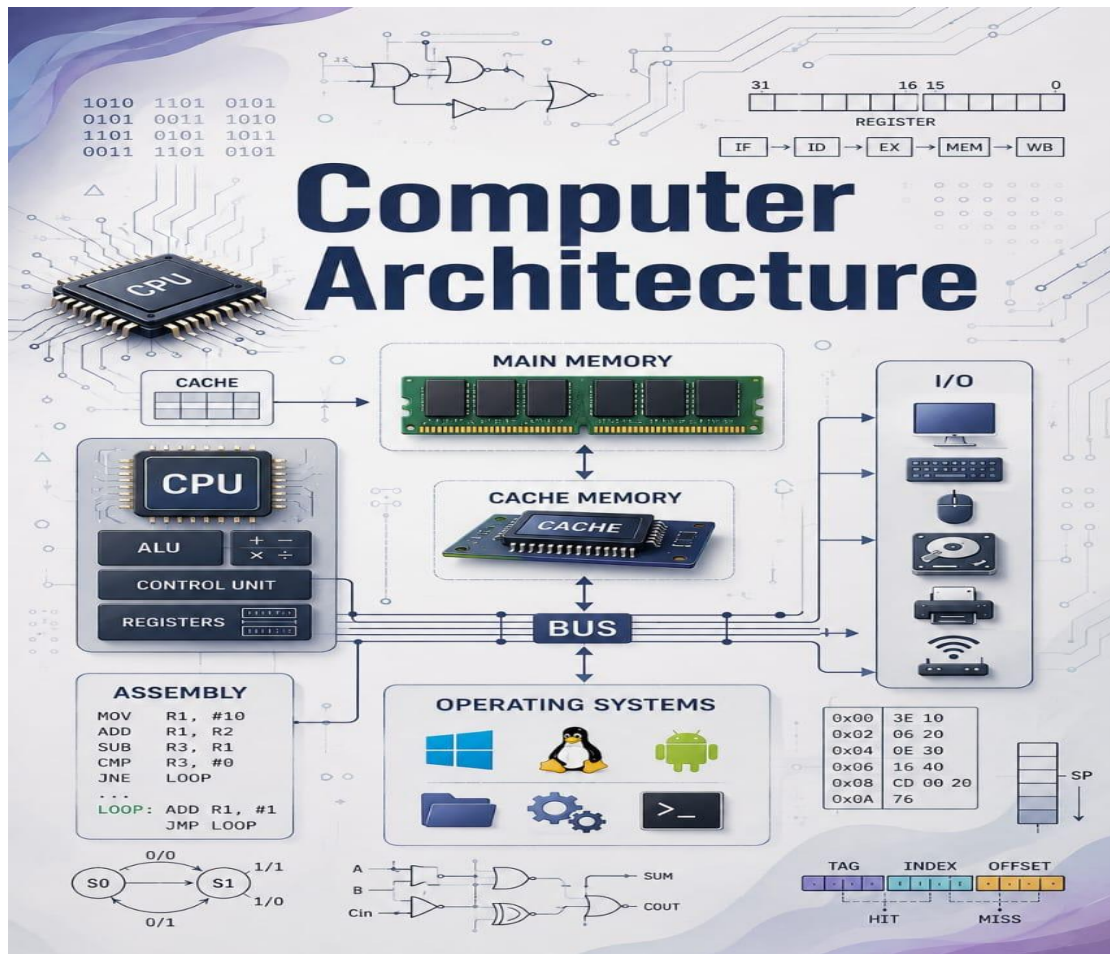




Computer Architecture

Content:

Chapter 01 : Basic Concepts and Compiler Evolution	Page
	1
Chapter 02 : Performance Issues	7
Chapter 03 : A Top-Level View of Computer Function and Interconnection	12
Chapter 04 : Cache Memory	18
Chapter 05 : Internal Memory	28
Chapter 06 : Input/Output	32
Chapter 07 : Operating System Support	38
Chapter 08 : Instruction Sets : Addressing	43
Chapter 09 : Assembly	48

Chapter 01 :Basic Concepts and Computer Evolution

01*Architecture & Organization:

- Architecture is those attributes visible to the programmer
- Instruction set, number of bits used for data representation, I/O mechanisms, addressin techniques.
- e.g. Is there a multiply instruction?
- Organization is how features are implemented
- Control signals, interfaces, memory technology.
- e.g. Is there a hardware multiply unit or is it done by repeated addition?
- All Intel x86 family share the same basic architecture
- The IBM System/370 family share the same basic architecture
- This gives code compatibility
- At least backwards
- Organization differs between different versions

02*Structure and Function:

Structure:

- The way in which components relate to each other
- Function
- The operation of individual components as part of the structure

Function:

- There are four basic functions that a computer can perform:
- Data processing
- Data may take a wide variety of forms and the range of processing requirements is broad
- Data storage
- Short-term
- Long-term
- Data movement
- Input-output (I/O) - when data are received from or delivered to a device (peripheral) that is directly connected to the computer
- Data communications – when data are moved over longer distances, to or from a remote device

Control:

A control unit manages the computer's resources and orchestrates the performance of its functional parts in response

03*There are four main structural components of the computer:

- .CPU – controls the operation of the computer and performs its data processing functions
- .Main Memory – stores data
- .I/O – moves data between the computer and its external environment
- .System Interconnection – some mechanism that provides for communication among CPU, main memory, and I/O

04*CPU major structural component:

.Control Unit:

- Controls the operation of the CPU and hence the computer
- Arithmetic and Logic Unit (ALU)
- Performs the computer's data processing function

.Registers:

- Provide storage internal to the CPU

.CPU Interconnection:

- Some mechanism that provides for communication among the control unit, ALU, and registers

05*Multicore Computer Structure:

.Central processing unit (CPU):

- Portion of the computer that fetches and executes instructions
- Consists of an ALU, a control unit, and registers
- Referred to as a processor in a system with a single processing unit

.Core:

- An individual processing unit on a processor chip
- May be equivalent in functionality to a CPU on a single-CPU system
- Specialized processing units are also referred to as cores

.Processor:

- A physical piece of silicon containing one or more cores
- Is the computer component that interprets and executes instructions
- Referred to as a multicore processor if it contains multiple cores

06*Cache Memory:

- Multiple layers of memory between the processor and main memory
- Is smaller and faster than main memory
- Used to speed up memory access by placing in the cache data from main memory that is likely to be used in the near future
- A greater performance improvement may be obtained by using multiple levels of cache, with level 1 (L1) closest to the core and additional levels (L2, L3, etc.) progressively farther from the core

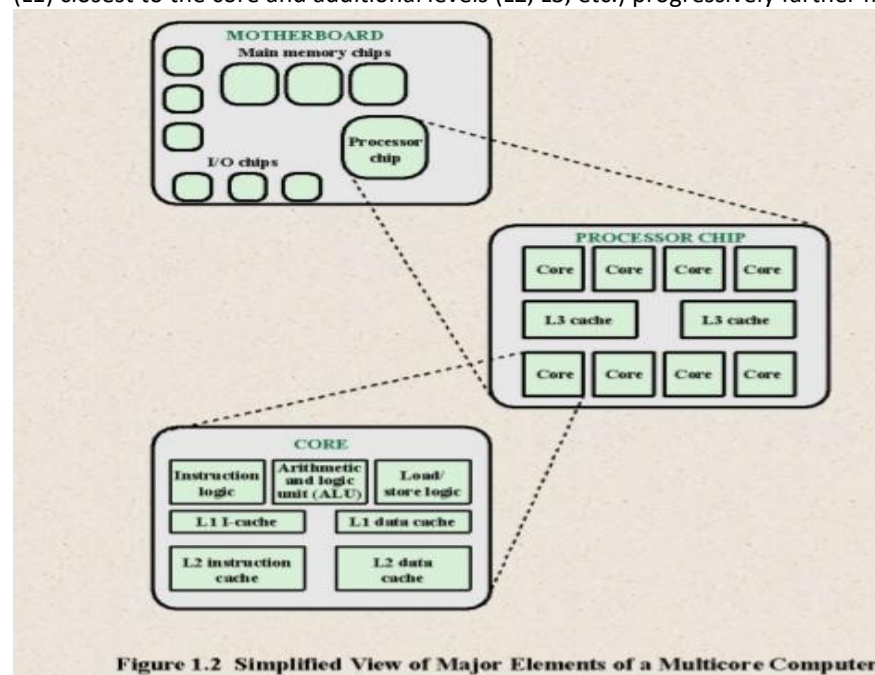


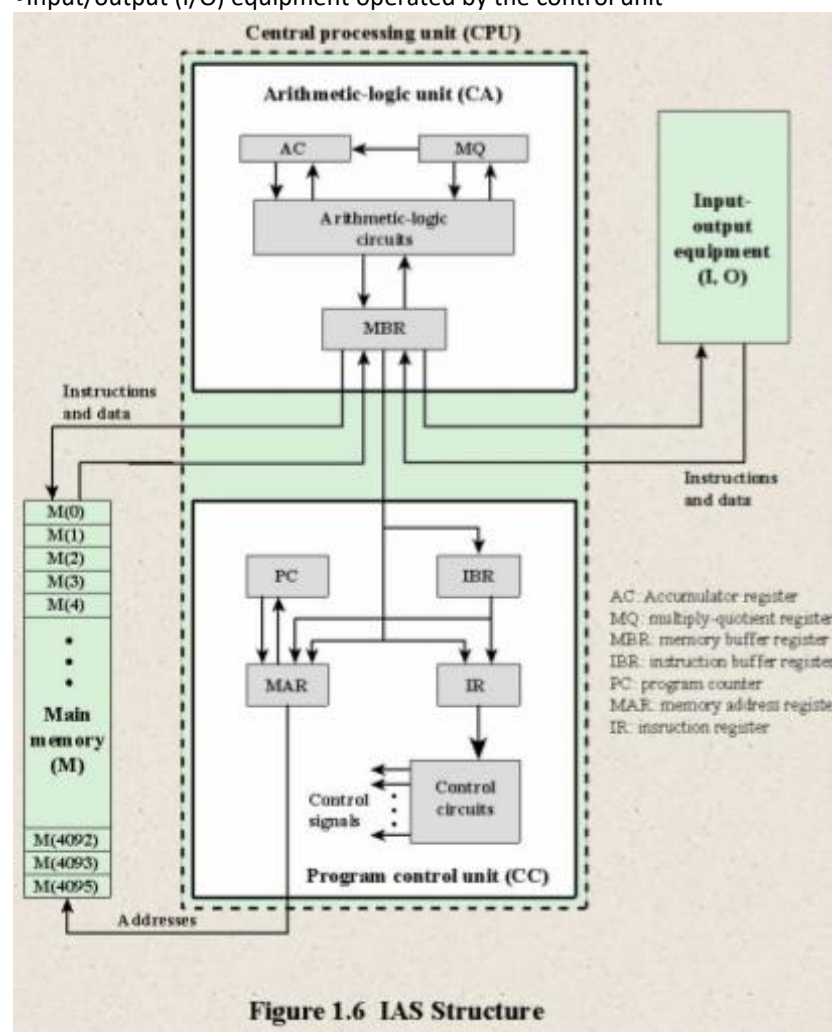
Figure 1.2 Simplified View of Major Elements of a Multicore Computer

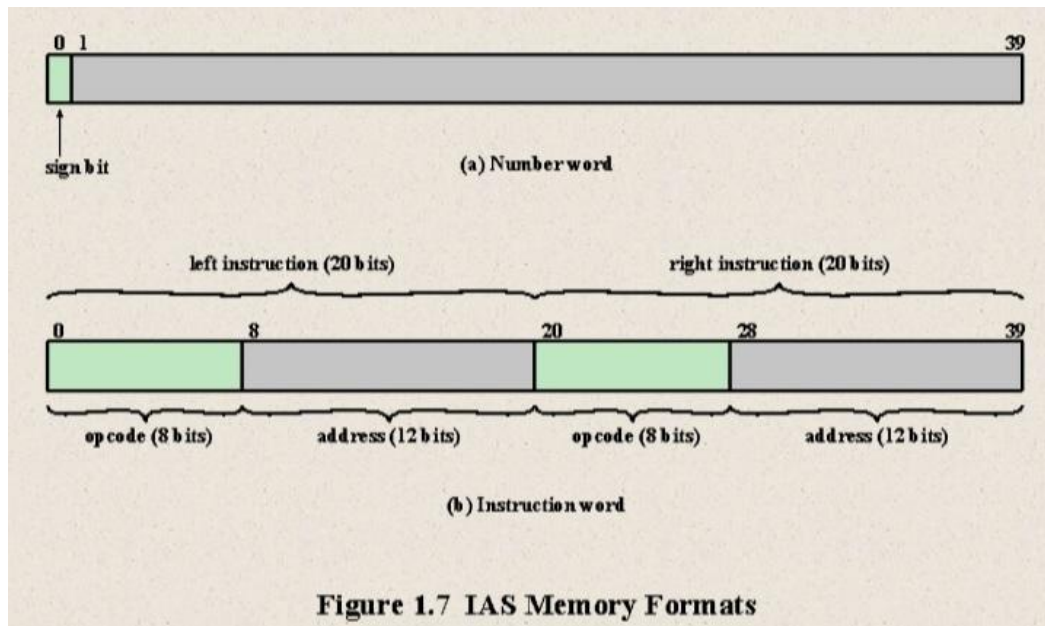
07*History of Computers:

.First Generation: Vacuum Tubes:

- Vacuum tubes were used for digital logic elements and memory
- IAS computer
- Fundamental design approach was the stored program concept
- Attributed to the mathematician John von Neumann
- First publication of the idea was in 1945 for the EDVAC
- Design began at the Princeton Institute for Advanced Studies
- Completed in 1952
- Prototype of all subsequent general-purpose computers

- A main memory, which stores both data and instructions
- An arithmetic and logic unit (ALU) capable of operating on binary data
- A control unit, which interprets the instructions in memory and causes them to be executed
- Input/output (I/O) equipment operated by the control unit





08*Registers:

Memory buffer register (MBR):

- .Contains a word to be stored in memory or sent to the I/O unit
- .Or is used to receive a word from memory or from the I/O unit

Memory address register (MAR):

- .Specifies the address in memory of the word to be written from or read into the MBR

Instruction register (IR):

- .Contains the 8-bit opcode instruction being executed

Instruction buffer register (IBR):

- .Employed to temporarily hold the right-hand instruction from a word in memory

Program counter (PC):

- .Contains the address of the next instruction pair to be fetched from memory

Accumulator (AC) and multiplier quotient (MQ):

- .Employed to temporarily hold operands and results of ALU operation

09*History of Computers:

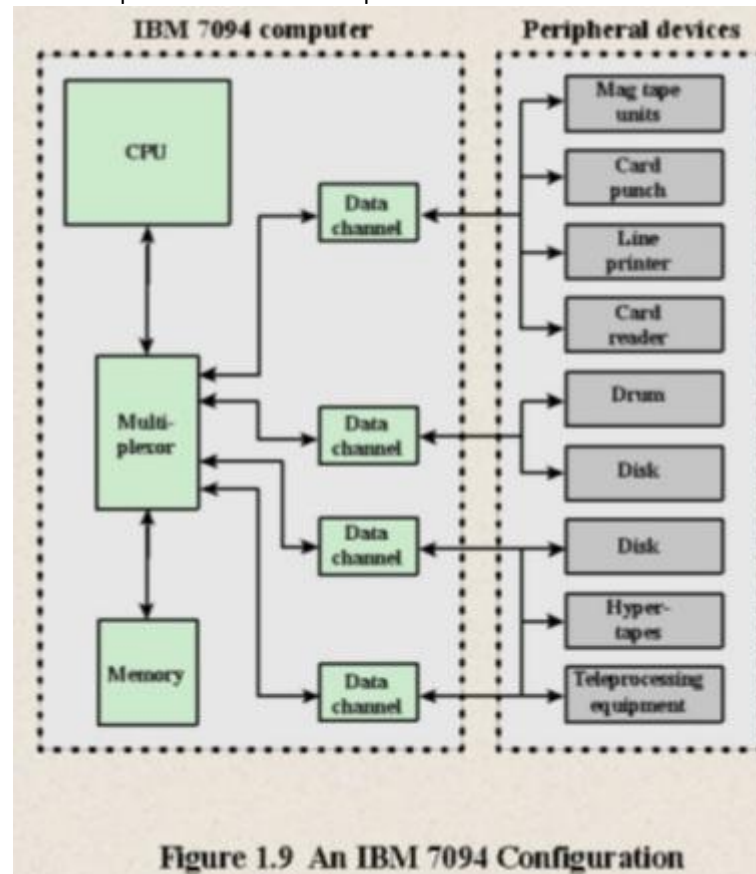
Second Generation: Transistors

- Smaller
- Cheaper
- Dissipates less heat than a vacuum tube
- Is a solid state device made from silicon
- Was invented at Bell Labs in 1947
- It was not until the late 1950's that fully transistorized computers were commercially available

Second Generation Computers:

.Introduced:

- More complex arithmetic and logic units and control units
- The use of high-level programming languages
- Provision of system software which provided the ability to:
 - Load programs
 - Move data to peripherals
 - Libraries perform common computations



Discrete component:

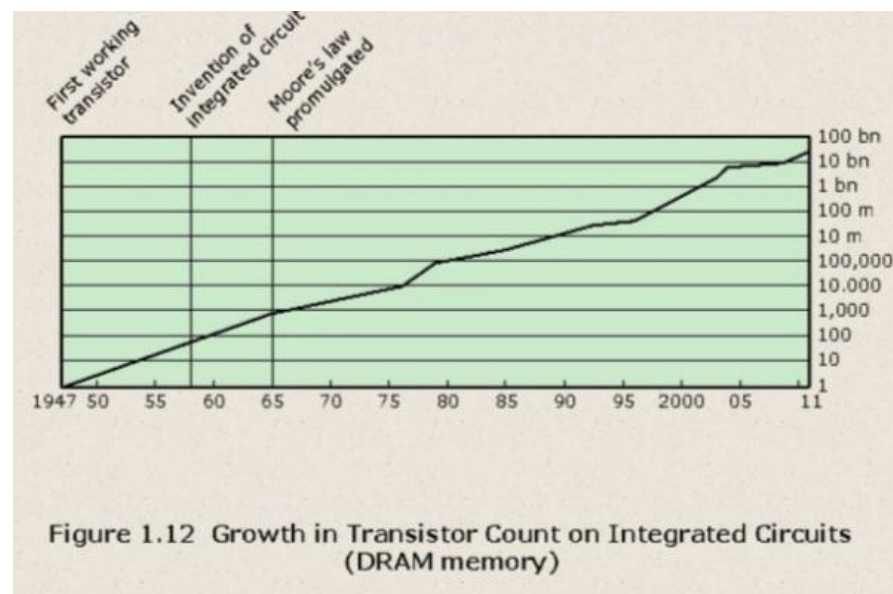
- Single, self-contained transistor
- Manufactured separately, packaged in their own containers, and soldered or wired together onto masonite-like circuit boards
- Manufacturing process was expensive and cumbersome
- The two most important members of the third generation were the IBM System/360 and the DEC PDP-8

10*Moore's Law:

- Gordon Moore – co-founder of Intel
- Observed number of transistors that could be put on a single chip was doubling every year
- Since 1970's development has slowed a little
- Number of transistors doubles every 18 months

.Consequences of Moore's law:

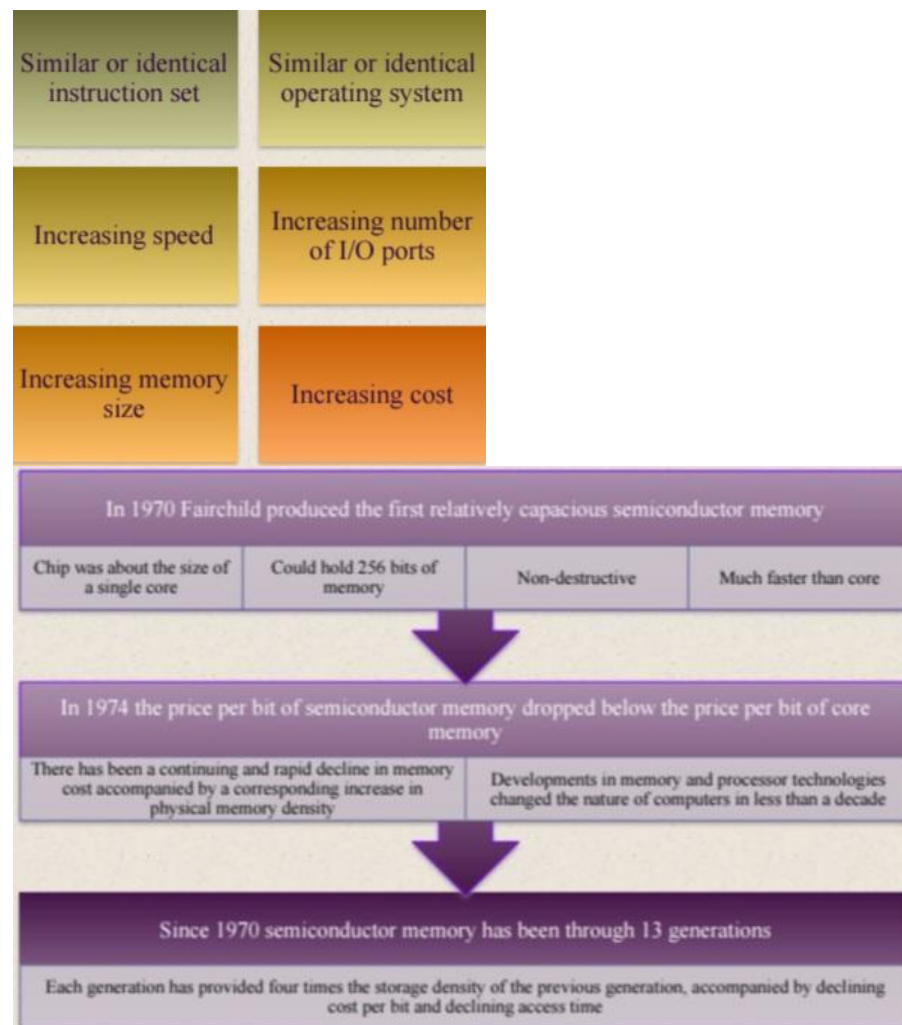
- The cost of computer logic and memory circuitry has fallen at a dramatic rate
- The electrical path length is shortened, increasing operating speed
- Computer becomes smaller and is more convenient to use in a variety of environments
- Reduction in power and cooling requirements
- Fewer interchip connections: increases reliability



11*IBM System/360:

- Announced in 1964
- Product line was incompatible with older IBM machines
- Was the success of the decade and cemented IBM as the overwhelmingly dominant computer vendor
- The architecture remains to this day the architecture of IBM's mainframe computers
- Was the industry's first planned family of computers
- Models were compatible in the sense that a program written for one model should be capable of being executed by another model in the series

Family Characteristics:



Chapter 02 : Performance Issues

01*Designing for Performance:

- The cost of computer systems continues to drop dramatically, while the performance and capacity of those systems continue to rise equally dramatically
- Today's laptops have the computing power of an IBM mainframe from 10 or 15 years ago
- Processors are so inexpensive that we now have microprocessors we throw away
- Desktop applications that require the great power of today's microprocessor-based systems include:
 - .Image processing
 - .Three-dimensional rendering
 - .Speech recognition
 - .Videoconferencing
 - .Multimedia authoring
 - .Voice and video annotation of files
 - .Simulation modeling
- Businesses are relying on increasingly powerful servers to handle transaction and database processing and to support massive client/server networks that have replaced the huge mainframe computer centers of yesteryear
- Cloud service providers use massive high-performance banks of servers to satisfy high-volume, high-transaction-rate applications for a broad spectrum of clients

Solution: Performance

Balance :

Increase the number of bits that are retrieved at one time by making

DRAMs “wider” rather than “deeper” and by using wide bus data paths

Change the DRAM interface to make it more efficient by including a cache or other buffering scheme on the DRAM chip

Reduce the frequency of memory access by incorporating increasingly complex and efficient cache structures between the processor and main memory

Increase the interconnect bandwidth between processors and memory by using higher speed buses and a hierarchy of buses to buffer and structure data flow

-Adjust the organization and architecture to compensate for the mismatch among the capabilities of the various components

-Architectural examples include:

I/O Devices:

Peripherals with intensive I/O demands

-Large data throughput demands

-Processors can handle this

-Problem moving data

02*Improvements in Chip:

Organization and Architecture:

-Increase hardware speed of processor

-Fundamentally due to shrinking logic gate size

-More gates, packed more tightly, increasing clock rate

-Propagation time for signals reduced

-Increase size and speed of caches

-Dedicating part of processor chip

-Cache access times drop significantly

-Change processor organization and architecture

-Increase effective speed of instruction execution

-Parallelism

03*Problems with Clock Speed and Logic Density:

.Power:

-Power density increases with density of logic and clock speed

-Dissipating heat

-RC-Resistive-capacitive delay

-Speed at which electrons flow limited by resistance and capacitance of metal wires connecting them

-Delay increases as the RC product increases

-As components on the chip decrease in size, the wire interconnects become thinner, increasing resistance

-Also, the wires are closer together, increasing capacitance

-Memory latency

-Memory speeds lag processor speeds

Solution: More emphasis on organizational and architectural approaches

.Increased Cache Capacity:

-Typically two or three levels of cache between processor and main memory

-Chip density increased

-More cache memory on chip

-Faster cache access

-Pentium chip devoted about 10% of chip area to cache

-Pentium 4 devotes about 50%

04*More Complex Execution Logic:

- Enable parallel execution of instructions
- Pipeline works like assembly line
- The execution of an instruction involves multiple stages of operation (fetch the instruction , decode , fetch operand , perform calculation ...)
- Pipelining enables a processor to work simultaneously on multiple instructions by performing a different phase for each of the multiple instructions at the same time.
- Superscalar allows multiple pipelines within single processor
- Instructions that do not depend on one another can be executed in parallel

05*Diminishing Returns:

- Internal organization of processors complex
- Can get a great deal of parallelism
- Further significant increases likely to be relatively modest
- Benefits from cache are reaching limit
- Increasing clock rate runs into power dissipation problem
- Some fundamental physical limits are being reached

06*Multicore:

- The use of multiple processors on the same chip provides the potential to increase performance without increasing the clock rate
- Strategy is to use two simpler processors on the chip rather than one more complex processor
- With two processors larger caches are justified
- As caches became larger it made performance sense to create two and then three levels of cache on a chip

07*Many Integrated Core (MIC) and Graphics Processing Unit (GPU):

MIC:

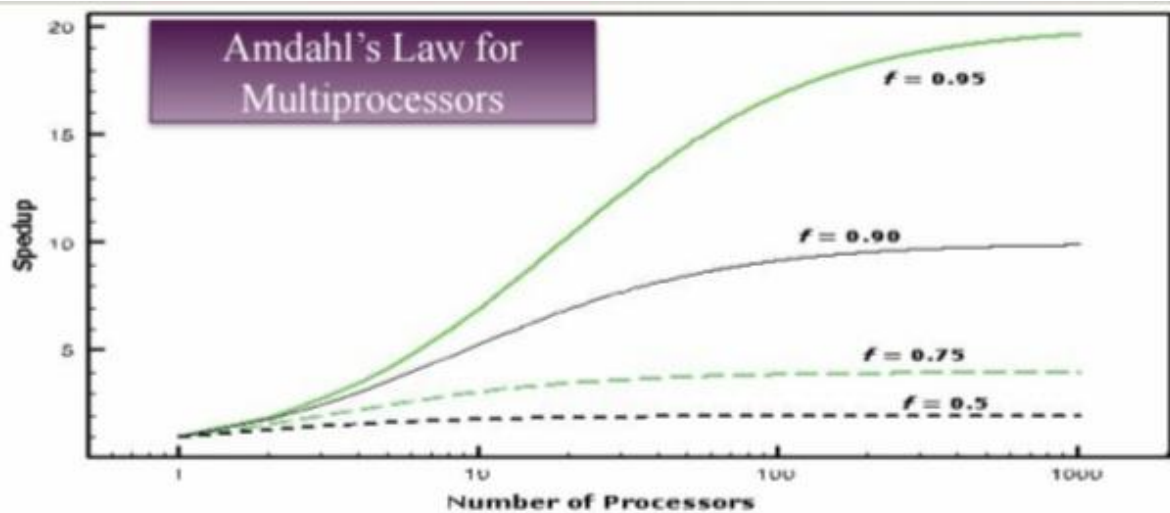
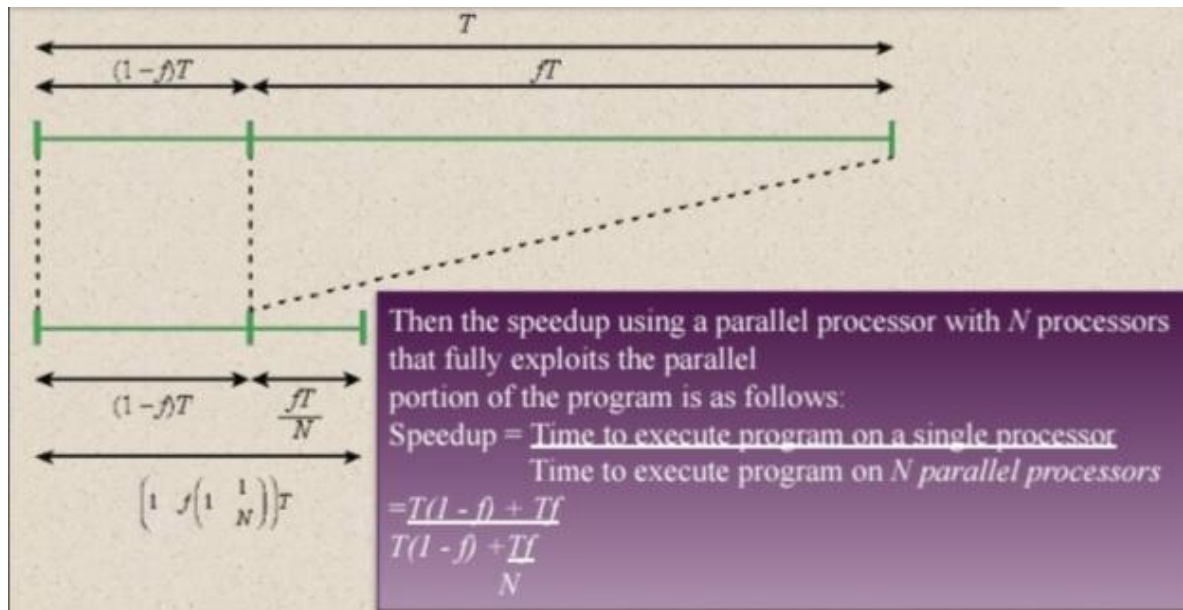
- Leap in performance as well as the challenges in developing software to exploit such a large number of cores
- The multicore and MIC strategy involves a homogeneous collection of general purpose processors on a single chip

GPU:

- Core designed to perform parallel operations on graphics data
- Traditionally found on a plug-in graphics card, it is used to encode and render 2D and 3D graphics as well as process video
- Used as vector processors for a variety of applications that require repetitive computations

08*Amdahl's Law:

- was first proposed by Gene Amdahl
- Deals with the potential speedup of a program using multiple processors compared to a single processor
- a speedup in one aspect of the technology or design does not necessarily result in a corresponding improvement in performance
- Illustrates the problems facing industry in the development of multi-core machines
- Software must be adapted to a highly parallel execution environment to exploit the power of parallel processing
- Can be generalized to evaluate and design technical improvement in a computer system



Two important conclusions can be drawn:

1. When f is small, the use of parallel processors has little effect.
2. As N approaches infinity, speedup is bound by $1/(1-f)$, so that there are diminishing returns for using more processors. These conclusions are too pessimistic. For example, a server can maintain multiple threads or multiple tasks to handle multiple clients and execute the threads or tasks in parallel up to the limit of the number of processors. Many database applications involve computations on massive amounts of data that can be split up into multiple parallel tasks.

09*Performance Assessment:

Clock Speed:

- Operations performed by a processor, such as fetching an instruction, decoding the instruction, performing an arithmetic operation, and so on, are governed by a system clock.
- The speed of a processor is dictated by the pulse frequency produced by the clock, measured in cycles per second, or Hertz (Hz).
- A 1-GHz processor receives 1 billion pulses per second. The rate of pulses is known as the clock rate, or clock speed.
- One increment, or pulse, of the clock is referred to as a clock cycle, or a clock tick. The time between pulses is the cycle time.
- Most instructions on most processors require multiple clock cycles to complete.
- Some instructions may take only a few cycles, while others require dozens.
- When pipelining is used, multiple instructions are being executed simultaneously.

-A straight comparison of clock speeds on different processors does not tell the whole story about performance.

Instruction Execution Rate:

$$\text{MIPS} = \text{Instruction Count} / (\text{Execution Time} \times 10^6)$$

Or

$$\text{MIPS} = \text{Clock Rate} / (\text{CPI} \times 10^6)$$

$$\text{CPI} = \text{CPU Cycles} / \text{Instruction Count}$$

Or

$$\text{CPI} = \text{Execution Time} \times \text{Clock Rate} / \text{Instruction Count}$$

10*System Performance Evaluation Corporation (SPEC):

$$\text{Execution time} = \text{CPU Clock Cycles} / \text{Clock Rate}$$

Or

$$\text{Execution Time} = \# \text{ nb of cycles of a Program} / \text{Clock Rate (cycles/Second)}$$

Or

$$\text{Execution Time} = \text{CPU Clock Cycles} \times \text{Clock Cycle Time}$$

11*SPEC CPU2006:

-Best known SPEC benchmark suite

-Industry standard suite for processor intensive applications

-Appropriate for measuring performance for applications that spend most of their time doing computation rather than I/O

-Consists of 17 floating point programs written in C, C++, and Fortran and 12 integer programs written in C and C++

-Suite contains over 3 million lines of code

-Fifth generation of processor intensive suites from SPEC

Chapter 03 : A Top-Level View of Computer Function and Interconnection

01*Computer Components:

-Contemporary computer designs are based on the von Neumann architecture :

-Data and instructions are stored in a single read-write memory

-The contents of this memory are addressable by location, without regard to the type of data contained there

Execution occurs in a sequential fashion (unless explicitly modified) from one instruction to the next

02*Hardware and Software Approaches:

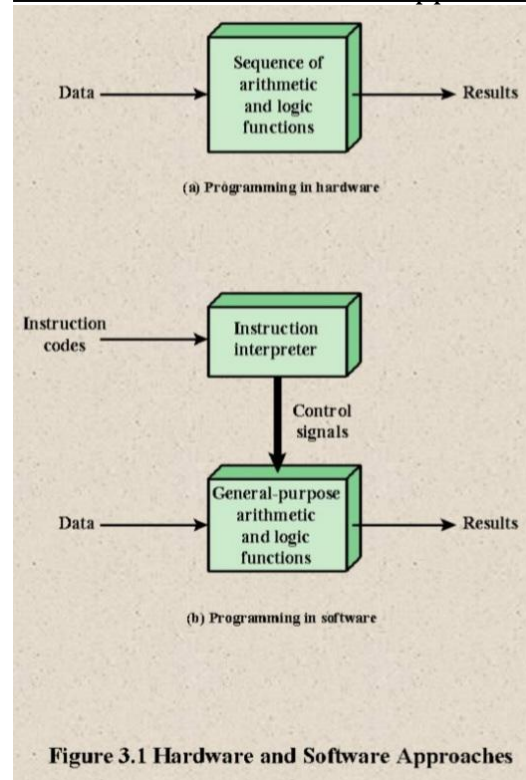


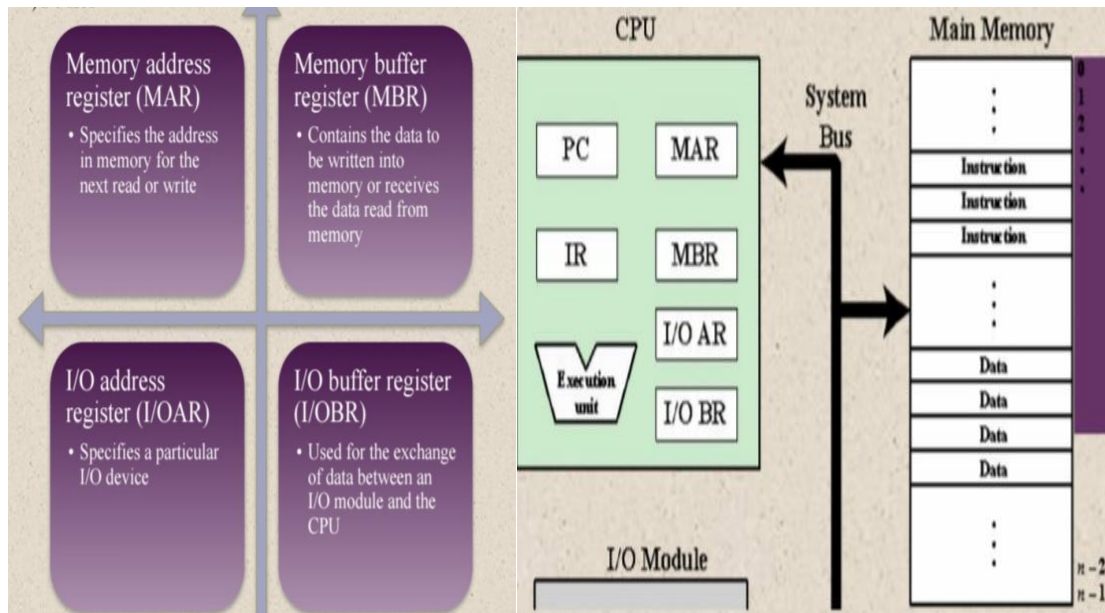
Figure 3.1b indicates two major components of the system: an instruction interpreter and a module of general-purpose arithmetic and logic functions. These two constitute the CPU.

Software:

- A sequence of codes or instructions
- Part of the hardware interprets each instruction and generates control signals
- Provide a new sequence of codes for each new program instead of rewiring the hardware

Major components for a functioning Computer:

- CPU
- Instruction interpreter
- Module of general-purpose arithmetic and logic functions
- I/O Components
- Input module
- Contains basic components for accepting data and instructions and converting them into an internal form of signals usable by the system
- Output module
- Means of reporting results



- PC: Program Counter
- IR: Instruction Register
- MAR (Memory Address Register): Specifies the address in memory for the next read or write
- MBR (Memory Buffer Register): Contains the data to be written into memory or receives the data read from memory
- I/OAR (Input/Output address register): Specifies a particular I/O device
- I/OBR (Input/Output Buffer Register): Used for the exchange of data between an I/O module and the CPU

The processing required for a single instruction is called an instruction cycle.

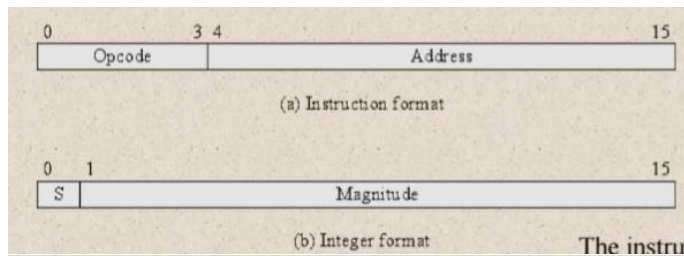
Program execution halts only if the machine is turned off, or some sort of unrecoverable error occurs, or a program instruction that halts the computer is encountered.

Fetch Cycle:

- At the beginning of each instruction cycle the processor fetches an instruction from memory
- The program counter (PC) holds the address of the instruction to be fetched next
- The processor increments the PC after each instruction fetch so that it will fetch the next instruction in sequence
- The fetched instruction is loaded into the instruction register (IR)
- The processor interprets the instruction and performs the required action

03*Action Categories:

- Processor-memory
- Data transferred from processor to memory or from memory to processor
- Processor-I/O
- Data transferred to or from a peripheral device by transferring between the processor and an I/O module
- Data processing
- The processor may perform some arithmetic or logic operation on data
- Control
- An instruction may specify that the sequence of execution be altered (for example: goto instruction)



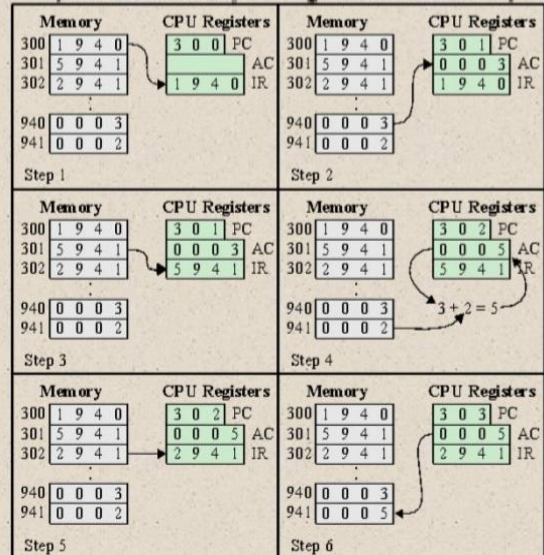
Program Counter (PC) = Address of instruction
 Instruction Register (IR) = Instruction being executed
 Accumulator (AC) = Temporary storage

(c) Internal CPU registers

0001 = Load AC from Memory
 0010 = Store AC to Memory
 0101 = Add to AC from Memory

(d) Partial list of opcodes

Figure 3.6 Example of Program Execution
 (contents of memory and registers in hexadecimal)

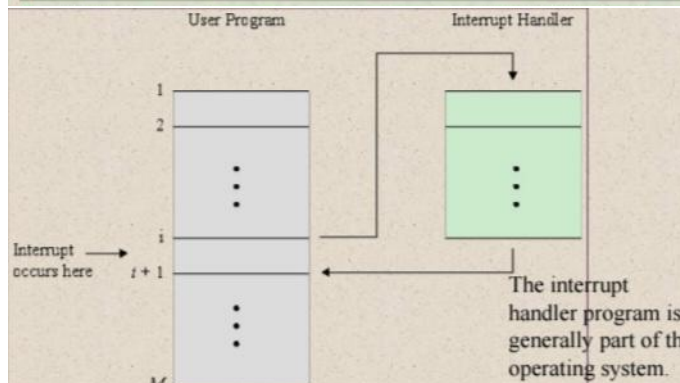


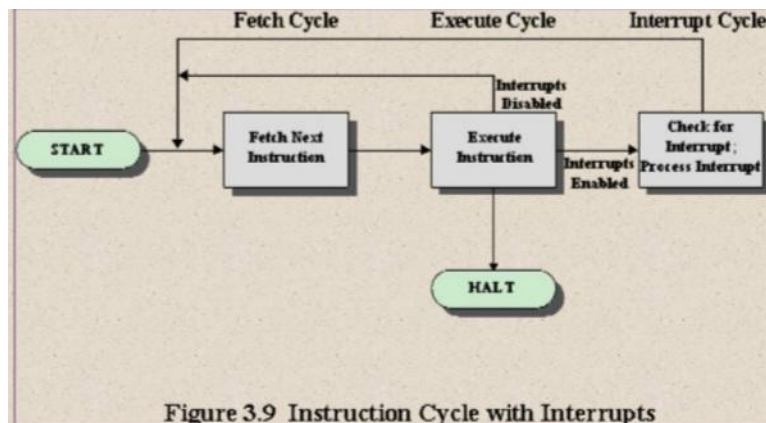
04*Interrupts:

All computers provide a mechanism by which other modules (I/O, memory) may interrupt the normal processing of the processor. Table 3.1 lists the most common classes of interrupts.

Table 3.1: Classes of Interrupts

Program	Generated by some condition that occurs as a result of an instruction execution, such as arithmetic overflow, division by zero, attempt to execute an illegal machine instruction, or reference outside a user's allowed memory space.
Timer	Generated by a timer within the processor. This allows the operating system to perform certain functions on a regular basis.
I/O	Generated by an I/O controller, to signal normal completion of an operation, request service from the processor, or to signal a variety of error conditions.
Hardware failure	Generated by a failure such as power failure or memory parity error.



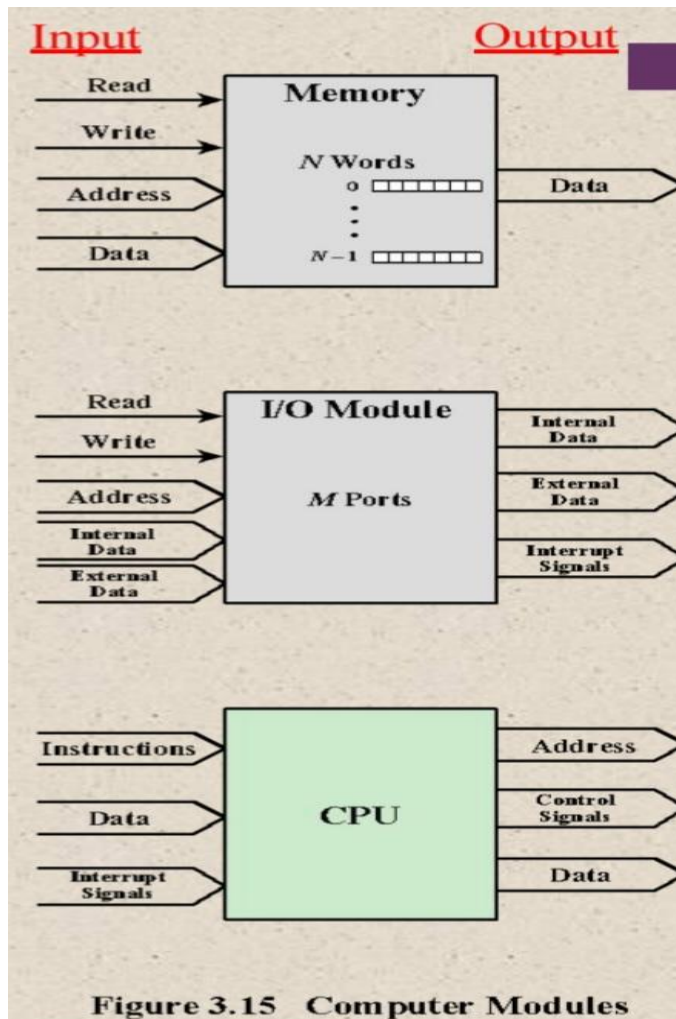


05*I/O Function:

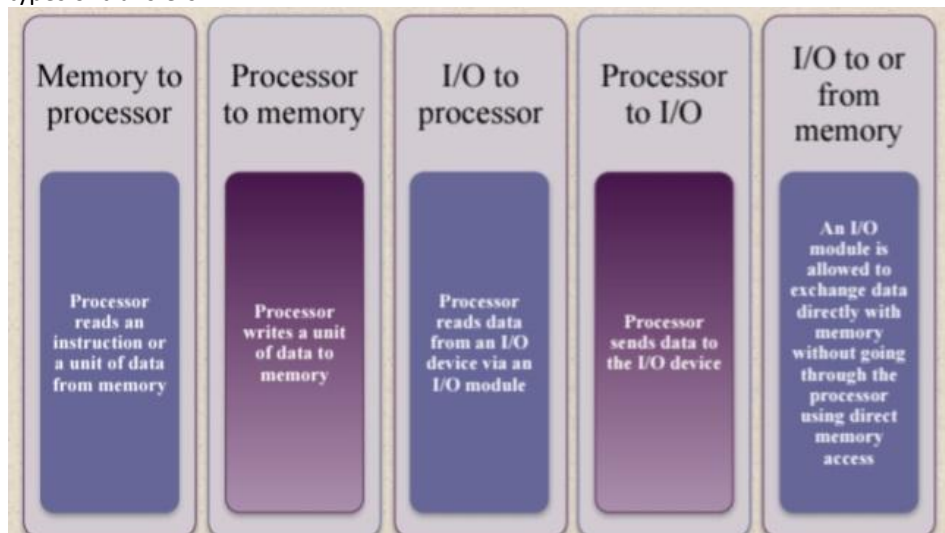
- I/O module can exchange data directly with the processor
- Processor can read data from or write data to an I/O module
- Processor identifies a specific device that is controlled by a particular I/O module
- I/O instructions rather than memory referencing instructions
- In some cases it is desirable to allow I/O exchanges to occur directly with memory
- The processor grants to an I/O module the authority to read from or write to memory so that the I/O memory transfer can occur without tying up the processor
- The I/O module issues read or write commands to memory relieving the processor of responsibility for the exchange
- This operation is known as direct memory access (DMA)

Types of data exchanges:

- Memory:
 - N words of equal length.
 - Each word is assigned a unique numerical address.
 - A word of data can be read from or written into the memory: read and write control signals. The location for the operation is specified by an address.
- I/O module:
 - There are two operations, read and write. Further, an I/O module may control more than one external device. We can refer to each of the interfaces to an external device as a port and give each a unique address.
 - In addition, there are external data paths for the input and output of data with an external device.
 - Finally, an I/O module may be able to send interrupt signals to the processor.
- Processor: The processor reads in instructions and data, writes out data after processing, and uses control signals to control the overall operation of the system. It also receives interrupt signals.

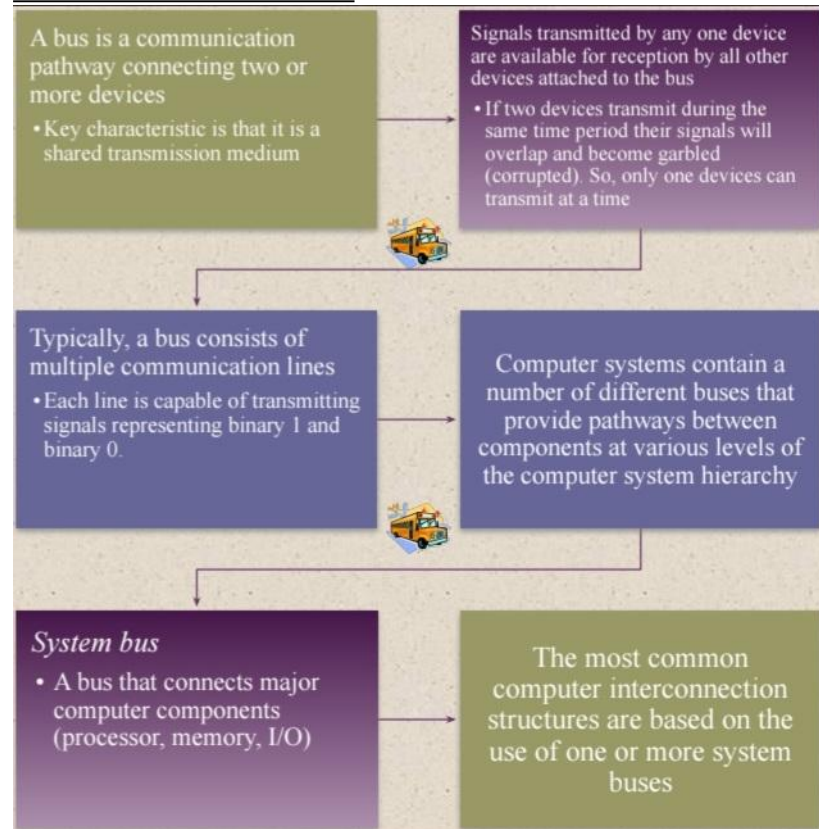


The interconnection structure (between cpu, memory and I/O Module) must support the following types of transfers:



By far the most common types of interconnection structures are
 (1) the bus
 (2) point-to-point interconnection structures .

06*Interconnection Bus:



07*Data Bus:

- The Data lines provide a path for moving data among system modules. The collection of these lines are called the data bus.
- The data bus may consist of 32, 64, 128, or more separate lines
- The number of lines is referred to as the width of the data bus
- Each line can carry only 1 bit at a time. The number of lines determines how many bits can be transferred at a time
- The width of the data bus is a key factor in determining overall system performance
- For example, if the data bus is 32 bits wide and each instruction is 64 bits long, then the processor must access the memory module twice during each instruction cycle.

08*Address Bus:

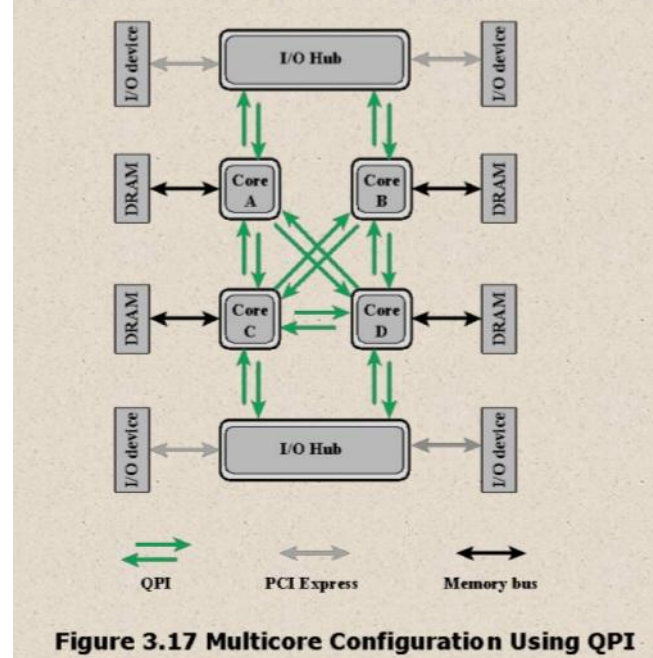
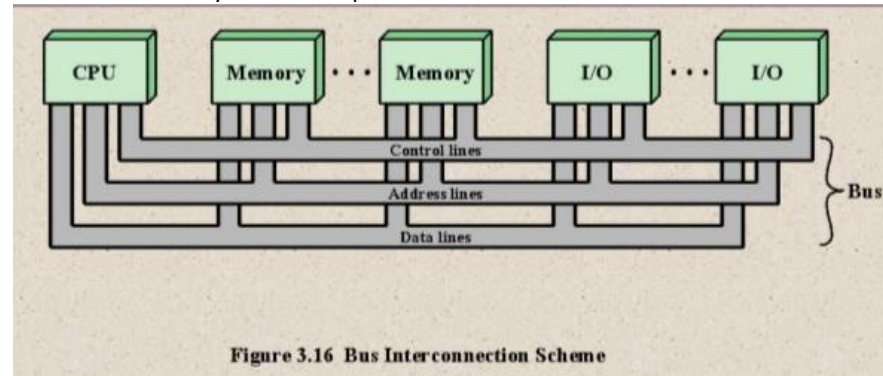
- Used to specify the source or destination of the data on the data bus
 - Example: If the processor wishes to read a word of data from memory it puts the address of the desired word on the address lines
 - The Width of the address bus determines the maximum possible memory capacity of the system
 - Address bus is Also used to address I/O ports
 - The higher order bits are used to select a particular module on the bus and the lower order bits select a memory location or I/O port within the module
- For example, on an 8-bit address bus, address 01111111 and below might reference locations in a memory module (module 0), and address 10000000 and above refer to devices attached to an I/O module (module 1).

09*Control Bus:

- Because the data and address lines are shared by all components there must be a means of controlling their use
- Control signals transmit both command and timing information among system modules
- Timing signals indicate the validity of data and address information
- Command signals specify operations to be performed

Typical control lines include:

- Memory write: causes data on the bus to be written into the addressed location
- Memory read: causes data from the addressed location to be placed on the bus
- I/O write: causes data on the bus to be output to the addressed I/O port
- I/O read: causes data from the addressed I/O port to be placed on the bus
- Clock: is used to synchronize operations



Chapter 04 : Cache Memory

01*Location:

- Location: refers to whether memory is internal and external to the computer.
- The typical computer system is equipped with a hierarchy of memory subsystems,
- some internal to the system (directly accessible by the processor): e.g. processors registers, cache, main memory
- and some external (accessible by the processor via an I/O module): e.g. optical disks, magnetic disks, tapes.

02*Capacity:

- For internal memory, this is typically expressed in terms of bytes (1 byte = 8 bits) or words. Common word lengths are 8, 16, and 32 bits.
- The word is the natural unit of organisation, and is equal to the number of bits to represent instruction (some times there is exceptions for that).
- For external memory capacity is typically expressed in terms of bytes.

03*Unit of Transfer:

- Internal : Usually governed by data bus width
- External : Usually a block which is much larger than a word
- Addressable unit : Smallest location which can be uniquely addressed
- Word internally : The relationship between the length in bits A of an address and the number N of addressable units is $2^A = N$.

04*Access Methods:

-Sequential :

- .Memory is organized into units of data called records
- .Start at the beginning and read through in order
- .Access time depends on location of data and previous location
- .e.g. tape

-Direct:

- .Individual blocks have unique address based on physical location
- .Access is by jumping to vicinity plus sequential search
- .Access time depends on location and previous location
- .e.g. disk

-Random:

- .Individual addresses identify locations exactly
- .Access time is independent of location or previous access
- .Any location can be selected at random and directly addressed and accessed
- .e.g. RAM

-Associative:

- .A word is retrieved based on a portion of its contents rather than its address
- .Each location has its own addressing mechanism and retrieval time is independent of location or previous access
- .e.g. cache

Performance:

-Access time (latency)

- For random-access memory it is the time it takes to perform a read or write operation
- For non-random-access memory it is the time it takes to position the read-write mechanism at the desired location

-Memory cycle time

- .Access time plus any additional time required before second access can commence
- .Additional time may be required for transients to die out on signal lines or to regenerate data if they are read destructively
- .Concerned with the system bus, not the processor

-Transfer rate:

- .The rate at which data can be transferred into or out of a memory unit
- .For random-access memory it is equal to $1/(\text{cycle time})$

-Physical type:

- .Semiconductor memory: RAM
- .Magnetic surface memory: Disk & Tape
- .Optical: CD & DVD
- .Magneto-optical

05*Physical Characteristics:

.Several physical characteristics of data storage are important:

.Volatile memory: Information is lost when electrical power is switched off

-Nonvolatile memory:

.Once recorded, information remains without deterioration until deliberately changed

.No electrical power is needed to retain information

.Magnetic-surface memories: Are nonvolatile

.Semiconductor memory: May be either volatile or nonvolatile

-Nonerasable memory:

.Cannot be altered, except by destroying the storage unit

.Semiconductor memory of this type is known as read-only memory (ROM)

Organization refers to the physical arrangement of bits to form words

-Memory Hierarchy:

.Design constraints on a computer's memory can be summed up by three questions:

.How much, how fast, how expensive

.There is a trade-off among capacity, access time, and cost

.Faster access time, greater cost per bit

.Greater capacity, smaller cost per bit

.Greater capacity, slower access time

.The way out of the memory dilemma is not to rely on a single memory component or technology, but to employ a memory hierarchy

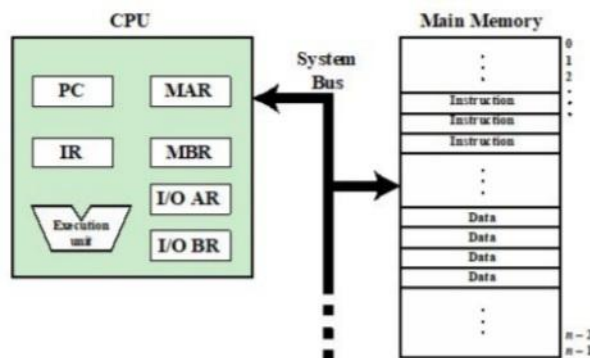
06*Cache Memory:

So you want fast?

.It is possible to build a computer which uses only static RAM

-This would be very fast

-This would cost a very large amount

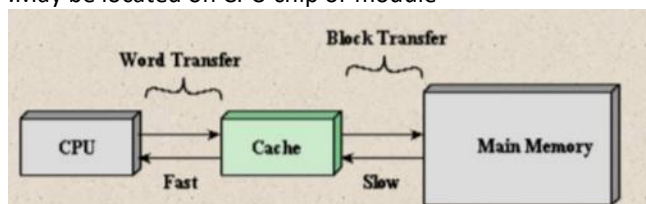


-Cache:

.Small amount of fast memory to improve the performance with cheap price.

.its between normal main memory and CPU to keep part of the data and instructions in the main memory.

.May be located on CPU chip or module



.Hit: when the processor fetch word and find it in the cache, otherwise Miss occurs.

.If miss occurs the block of the required word (consists of fixed number of words) is read from main memory and saved in the cache.

.Because of the locality principle, it likely to find the next referenced data or instructions in the cache as result of binging the whole block rather than the required word.

-Cache/Main Memory:

Structure

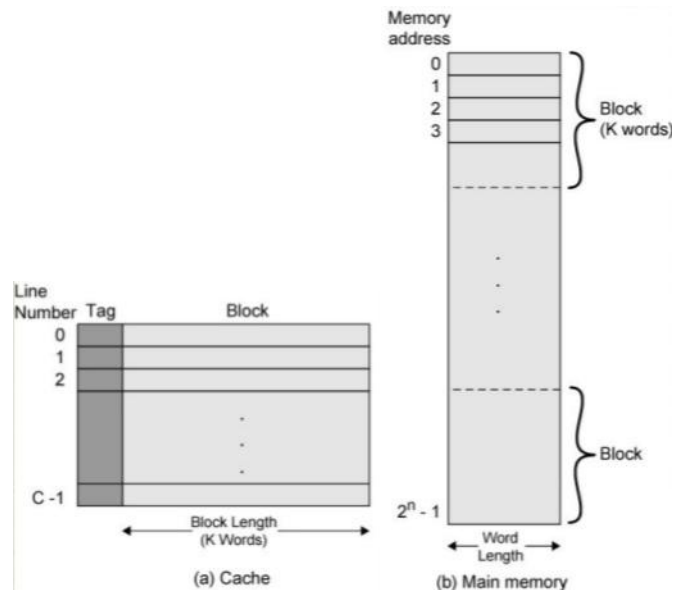
.Memory of size 2^n will have n address lines.

.The main memory is divided into blocks ,each block consists of k words (i.e. the number of blocks $M = 2^n/k$.

.The cache consists of lines C , each has the same size as block and $C \ll M$.

.At any time the cache will holds some blocks which are involved with processor work at that time.

.As the number of C is much less than the number of blocks, each line will have tag to indicate the block it holds.



-Cache Read Operation:

CPU requests contents of memory location

.Check cache for this data

.If present, get from cache (fast)

.If not present, read required block from main memory to cache

.Then deliver from cache to

-CPU:

.Cache includes tags to identify which block of main memory is in each cache slot

-Elements of Cache Design:

.Cache Address

.Size

.Mapping Function

.Replacement Algorithm

.Write Policy

.Block Size

.Number of Caches

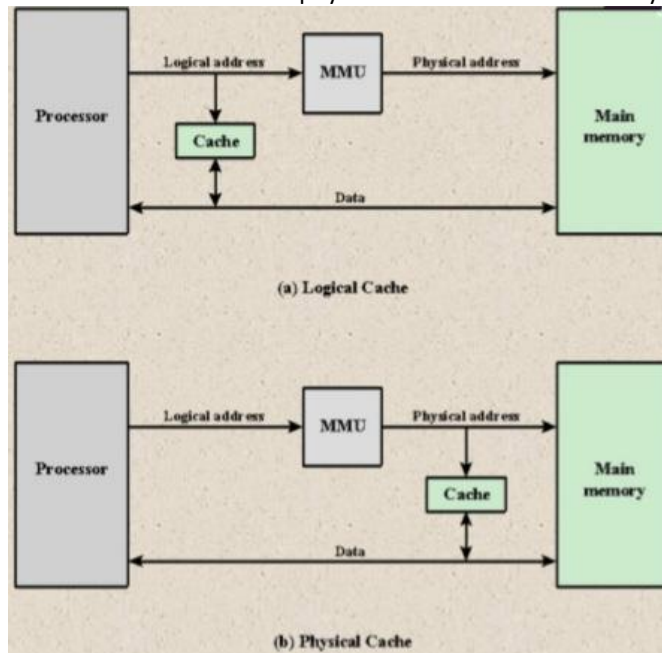
07*Cache Addresses - Visual memory:

.Virtual memory:

.Facility that allows programs to address memory from a logical point of view, without regard to the amount of main memory physically available

.When used, the address fields of machine instructions contain virtual addresses

.For reads from and writes to main memory, a hardware memory management unit (MMU) translates each virtual address into a physical address in main memory



When virtual addresses are used, the system designer may choose to place the cache between the processor and the MMU or between the MMU and main memory (Figure 4.7). A logical cache, also known as a virtual cache, stores data using virtual addresses. The processor accesses the cache directly, without going through the MMU. A physical cache stores data using main memory physical addresses.

-Size does matter:

.Cost

.More cache is more expensive

.Speed

.More cache is faster (up to a point)

-Checking cache for data takes time

Processor	Type	Year of Introduction	L1 Cache	L2 cache	L3 Cache
IBM 360/85	Mainframe	1968	16 to 32 kB	—	—
PDP-11/70	Minicomputer	1975	1 kB	—	—
VAX 11/780	Minicomputer	1978	16 kB	—	—
IBM 3033	Mainframe	1978	64 kB	—	—
IBM 3090	Mainframe	1985	128 to 256 kB	—	—
Intel 80486	PC	1989	8 kB	—	—
Pentium	PC	1993	8 kB/8 kB	256 to 512 KB	—
PowerPC 601	PC	1993	32 kB	—	—
PowerPC 620	PC	1996	32 kB/32 kB	—	—
PowerPC G4	PC/server	1999	32 kB/32 kB	256 KB to 1 MB	2 MB
IBM S/390 G6	Mainframe	1999	256 kB	8 MB	—
Pentium 4	PC/server	2000	8 kB/8 kB	256 KB	—
IBM SP	High-end server/ supercomputer	2000	64 kB/32 kB	8 MB	—
CRAY MTA	Supercomputer	2000	8 kB	2 MB	—
Itanium	PC/server	2001	16 kB/16 kB	96 KB	4 MB
Itanium 2	PC/server	2002	32 kB	256 KB	6 MB
IBM POWER5	High-end server	2003	64 kB	1.9 MB	36 MB
CRAY XD-1	Supercomputer	2004	64 kB/64 kB	1 MB	—
IBM POWER6	PC/server	2007	64 kB/64 kB	4 MB	32 MB
IBM z10	Mainframe	2008	64 kB/128 kB	3 MB	24-48 MB
Intel Core i7 EE 990	Workstation/ server	2011	6 × 32 kB/32 kB	1.5 MB	12 MB
IBM zEnterprise 196	Mainframe/ Server	2011	24 × 64 kB/ 128 kB	24 × 1.5 MB	24 MB L3 192 MB L4

08*Mapping Function:

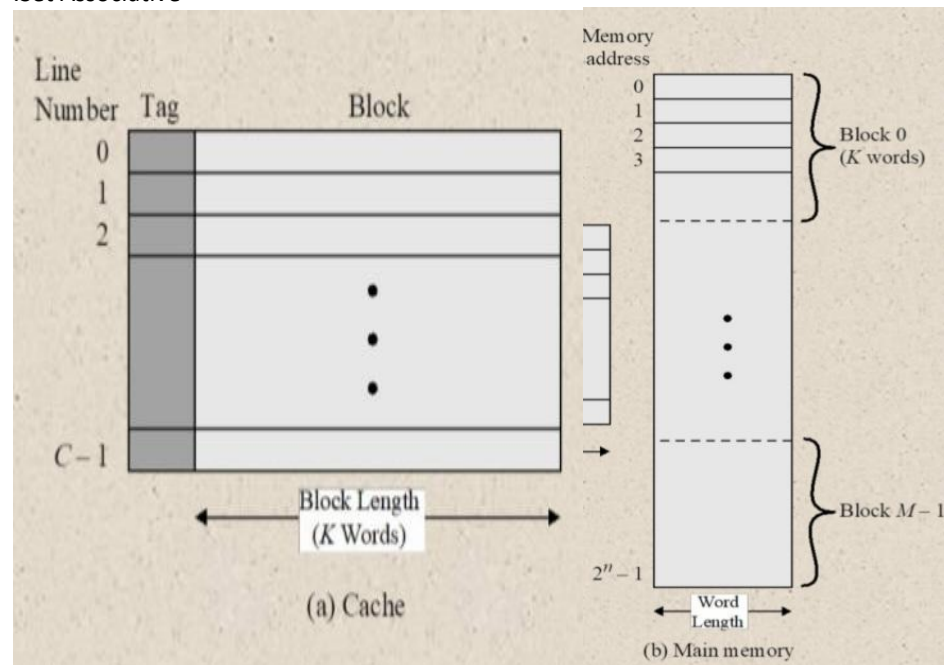
.Because there are fewer cache lines than main memory blocks, an algorithm is needed for mapping main memory blocks into cache lines

.Three techniques can be used:

.Direct

.Associative

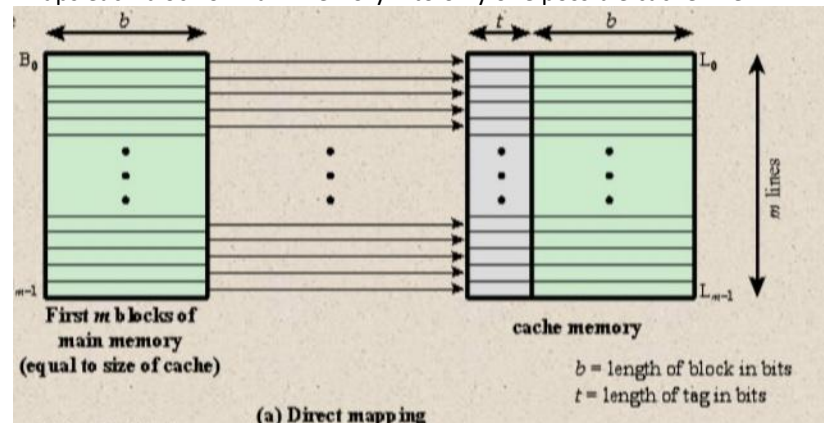
.Set Associative



-Direct Mapping:

.The simplest technique

.Maps each block of main memory into only one possible cache line

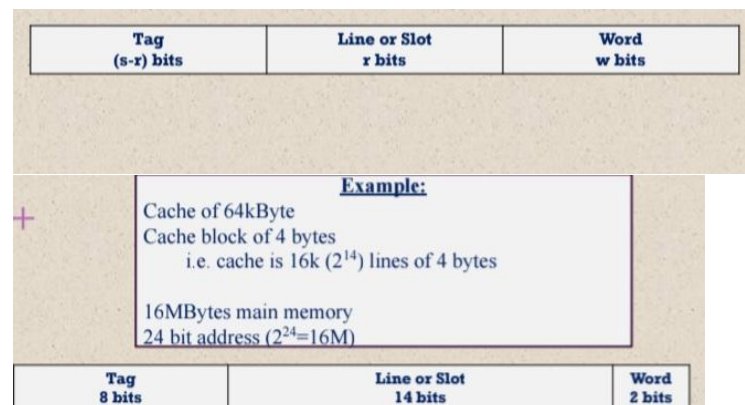


.Address is in two parts

.Least Significant w bits identify unique word (w depends on the block size)

.Most Significant s bits specify one memory block

.The MSBs are split into a cache line field r and a tag of $s-r$ (most significant)



24 bit address

2 bit word identifier (4 byte block)

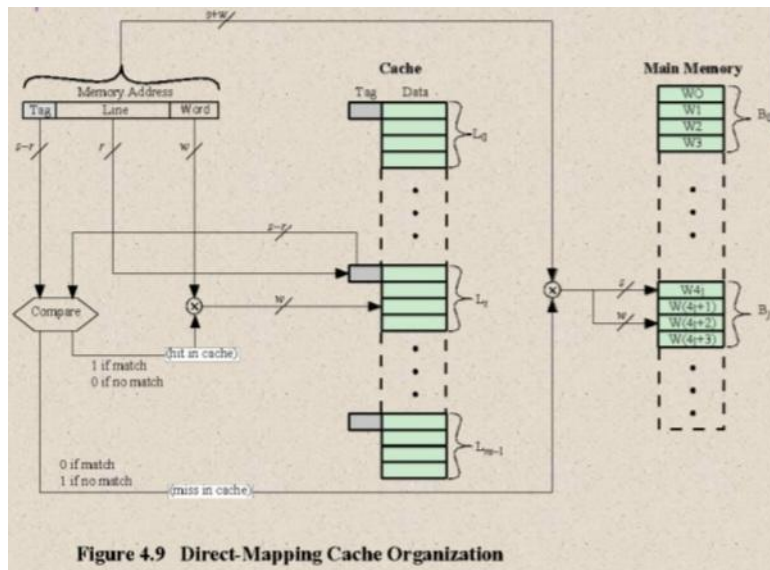
22 bit block identifier

8 bit tag (=22-14) [N.B. the no. of lines in the cache are 214= 16 K lines]

14 bit slot or line

No two blocks in the same line have the same Tag field

Check contents of cache by finding line and checking Tag



Address length = $(s + w)$ bits
 Number of addressable units = $2s+w$ words or bytes
 Block size = line size = $2w$ words or bytes
 Number of blocks in main memory = $2s+w/2w = 2s$
 Number of lines in cache = $m = 2r$
 Size of tag = $(s - r)$ bits

Direct Mapping pros & cons

Simple

Inexpensive

Fixed location for given block

If a program accesses 2 blocks that map to the same line repeatedly, cache misses are very high

09*Associative Mapping:

A main memory block can load into any line of cache

Memory address is interpreted as tag and word

Tag uniquely identifies block of memory

Every line's tag is examined for a match

Cache searching gets expensive

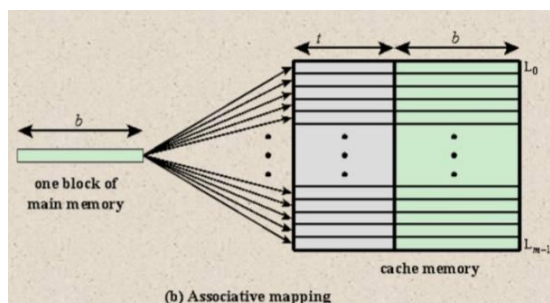


Figure 4.8 Mapping From Main Memory to Cache:

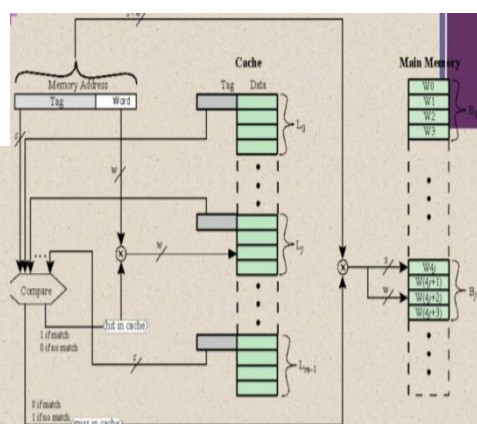


Figure 4.11 Fully Associative Cache Organization

Address length = $(s + w)$ bits
 Number of addressable units = 2^{s+w} words or bytes
 Block size = line size = 2^w words or bytes
 Number of blocks in main memory = $2^{s+w}/2^w = 2^s$
 Number of lines in cache = undetermined
 Size of tag = s bits

10*Set Associative Mapping:

Compromise that exhibits the strengths of both the direct and associative approaches while reducing their disadvantages

Cache consists of a number of sets

Each set contains a number of lines

A given block maps to any line in a given set

e.g. Block B can be in any line of set i

e.g. 2 lines per set

2 way associative mapping

A given block can be in one of 2 lines in only one set

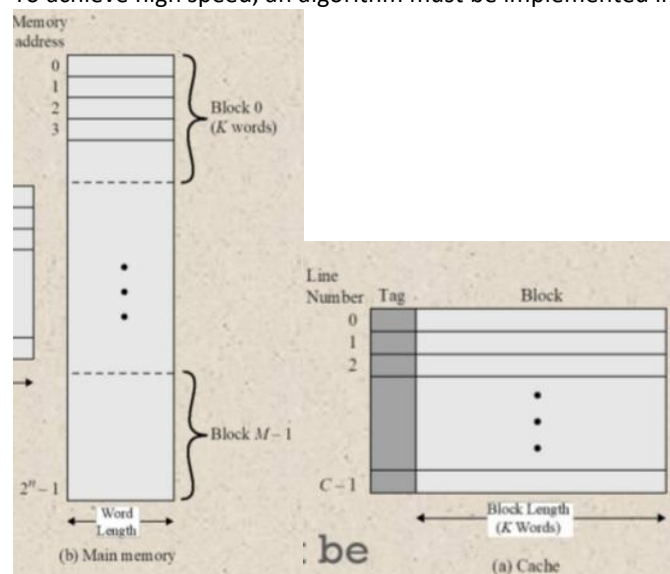
11*Replacement Algorithms:

Once the cache has been filled, when a new block is brought into the cache, one of the existing blocks must be replaced

For direct mapping there is only one possible line for any particular block and no choice is possible

For the associative and set-associative techniques a replacement algorithm is needed

To achieve high speed, an algorithm must be implemented in hardware



The most common replacement algorithms are:

- Least recently used (LRU)
- Most effective
- Replace that block in the set that has been in the cache longest with no reference to it
- Because of its simplicity of implementation, LRU is the most popular replacement algorithm
- e.g. in 2 way set associative
- Which of the 2 block is LRU? (use bit)
- First-in-first-out (FIFO)
- Replace that block in the set that has been in the cache longest
- Easily implemented as a round-robin or circular buffer technique
- Least frequently used (LFU)
- Replace that block in the set that has experienced the fewest references
- Could be implemented by associating a counter with each line

12*Write Policy:

- A cache line must not be overwritten by a block unless main memory is up to date
- Multiple CPUs may have individual caches
- I/O may address main memory directly and it may cause inconsistency.

13*Write Through and Write Back:

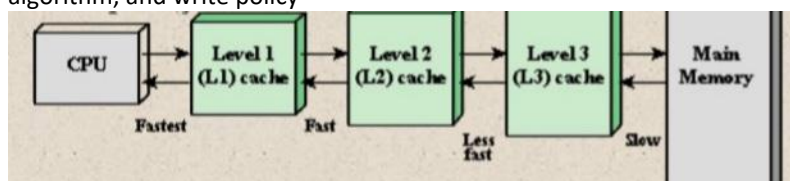
- Write through
 - .Simplest technique
 - .All write operations are made to main memory as well as to the cache
 - .The main disadvantage of this technique is that it generates substantial memory traffic and may create a bottleneck
- Write back
 - .Minimizes memory writes
 - .Updates are made only in the cache
 - .Portions of main memory are invalid and hence accesses by I/O modules can be allowed only through the cache
 - .This makes for complex circuitry and a potential bottleneck

14*Line Size:

- When a block of data is retrieved and placed in the cache not only the desired word but also some number of adjacent words are retrieved
- As the block size increases the hit ratio will at first increase because of the principle of locality
- As the block size increases more useful data are brought into the cache
- The hit ratio will begin to decrease as the block becomes bigger and the probability of using the newly fetched information becomes less than the probability of reusing the information that has to be replaced
- Two specific effects come into play:
 - Larger blocks reduce the number of blocks that fit into a cache
 - As a block becomes larger each additional word is farther from the requested word

15*Multilevel Caches:

- As logic density has increased it has become possible to have a cache on the same chip as the processor
- The on-chip cache reduces the processor's external bus activity and speeds up execution time and increases overall system performance
- When the requested instruction or data is found in the on-chip cache, the bus access is eliminated
- On-chip cache accesses will complete appreciably faster than would even zero-wait state bus cycles
- During this period the bus is free to support other transfers
- Two-level cache:
 - Internal cache designated as level 1 (L1)
 - External cache designated as level 2 (L2)
- Potential savings due to the use of an L2 cache depends on the hit rates in both the L1 and L2 caches
- The use of multilevel caches complicates all of the design issues related to caches, including size, replacement algorithm, and write policy



Unified Versus Split Caches:

- Has become common to split cache:
 - One dedicated to instructions
 - One dedicated to data

-Both exist at the same level, typically as two L1 caches

17*Advantages of unified cache:

- Higher hit rate
- Balances load of instruction and data fetches automatically. That is, if an execution pattern involves many more instruction fetches than data fetches, then the cache will tend to fill up with instructions, and if an execution pattern involves relatively more data fetches, the opposite will occur.
- Only one cache needs to be designed and implemented
- Trend is toward split caches at the L1 and unified caches for higher levels

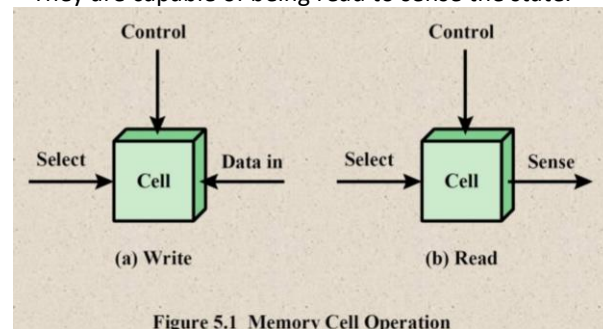
18*Advantages of split cache:

- Eliminates cache contention between instruction fetch/decode unit and execution unit
- Important in pipelining

Chapter 05 : Internal Memory

The basic element of a semiconductor memory is the memory cell. semiconductor memory cells share certain properties:

- They exhibit two states, which can be used to represent binary 1 and 0.
- They are capable of being written into (at least once), to set the state.
- They are capable of being read to sense the state.



01*Static RAM (SRAM):

- Digital device that uses the same logic elements used in the processor
- Binary values are stored using traditional flip-flop logic gate configurations
- Will hold its data as long as power is supplied to it

02*SRAM versus DRAM:

- Both volatile
- Power must be continuously supplied to the memory to preserve the bit values
- Dynamic cell
- Simpler to build, smaller
- More dense (smaller cells = more cells per unit area)
- Less expensive
- Requires the supporting refresh circuitry
- Tend to be favored for large memory requirements
- Used for main memory
- Static
- Faster
- Used for cache memory (both on and off chip)

03*Read Only Memory (ROM):

- Contains a permanent pattern of data that cannot be changed or added to
- No power source is required to maintain the bit values in memory (NON VOLATILE)
- Data or program is permanently in main memory and never needs to be loaded from a secondary storage device. (Usually, it is used for System Programs)
- Data is actually wired into the chip as part of the fabrication process (MASK)

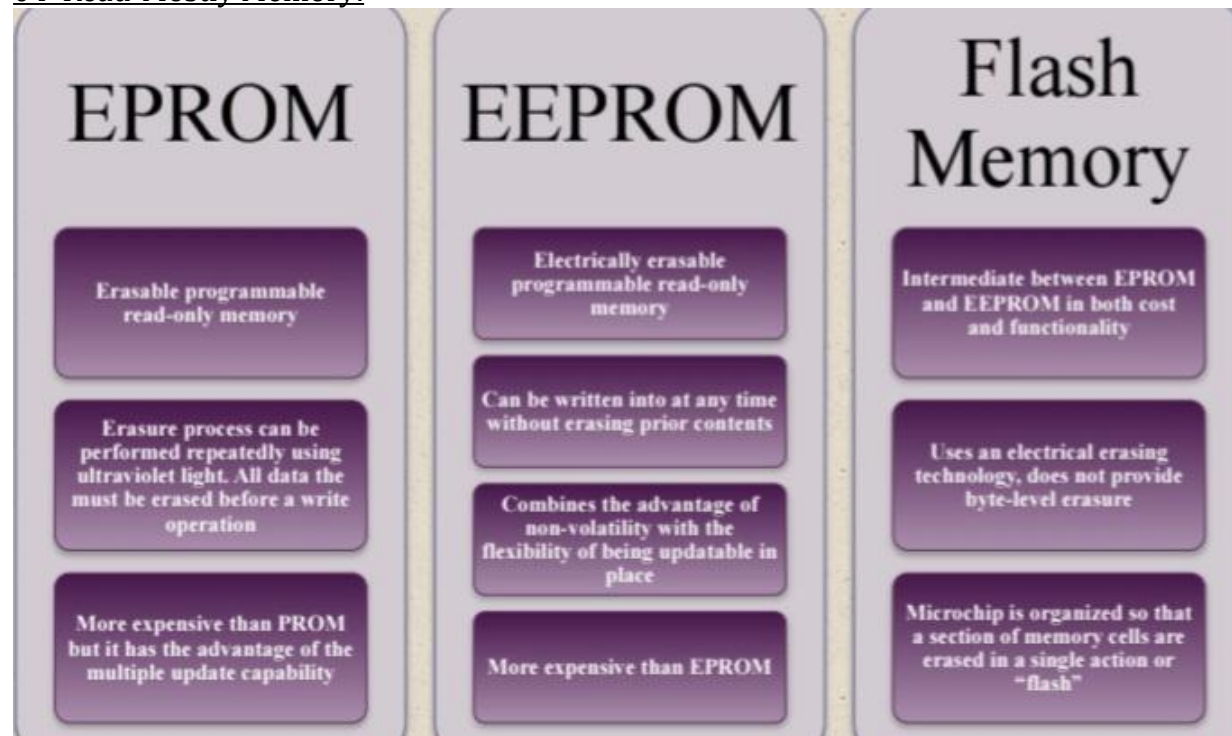
Disadvantages of this:

- No room for error, if one bit is wrong the whole batch of ROMs must be thrown out
- Data insertion step includes a relatively large fixed cost

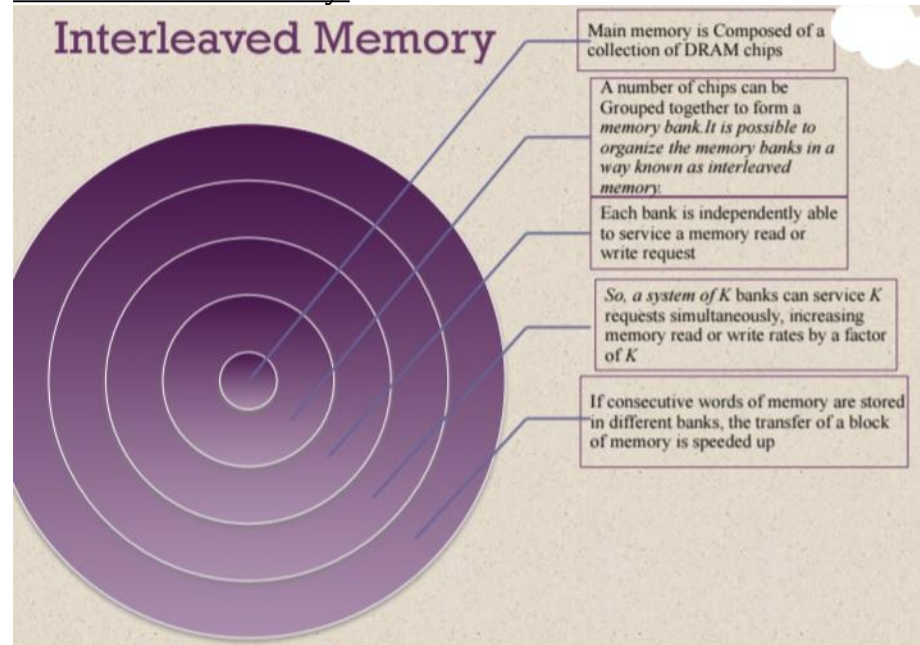
Programmable ROM (PROM):

- Less expensive alternative
- Nonvolatile and may be written into only once
- Writing process is performed electrically and may be performed by supplier or customer at a time later than the original chip fabrication
- Special equipment is required for the writing process
- Provides flexibility and convenience
- The ROM remains attractive for high volume production runs

04*Read-Mostly Memory:



05*Interleaved Memory:



06*Advanced DRAM Organization:

- One of the most critical system bottlenecks when using high-performance processors is the interface to main internal memory
- The traditional DRAM chip is constrained both by its internal architecture and by its interface to the processor's memory bus
- A number of enhancements to the basic DRAM architecture have been explored
- The schemes that currently dominate the market are SDRAM and DDR-DRAM

Synchronous DRAM (SDRAM):

One of the most widely used forms of DRAM

Exchanges data with the processor synchronized to an external clock signal and running at the full speed of the processor/memory bus without imposing wait states

With synchronous access the DRAM moves data in and out under control of the system clock

- The processor or other master issues the instruction and address information which is latched (fastened) by the DRAM
- The DRAM then responds after a set number of clock cycles
- Meanwhile the master can safely do other tasks while the SDRAM is processing

Double Data Rate SDRAM (DDR SDRAM):

-Developed by the JEDEC Solid State Technology Association

(Electronic Industries Alliance's semiconductor-engineering-standardization body)

-Numerous companies make DDR chips, which are widely used in desktop computers and servers

-DDR achieves higher data rates in three ways:

- First, the data transfer is synchronized to both the rising and falling edge of the clock, rather than just the rising edge. This doubles the data rate; hence the term double data rate
- Second, DDR uses higher clock rate on the bus to increase the transfer rate
- Third, a buffering scheme is used

	DDR1	DDR2	DDR3	DDR4
Prefetch buffer (bits)	2	4	8	8
Voltage level (V)	2.5	1.8	1.5	1.2
Front side bus data rates (Mbps)	200—400	400—1066	800—2133	2133—4266

Table 5.4

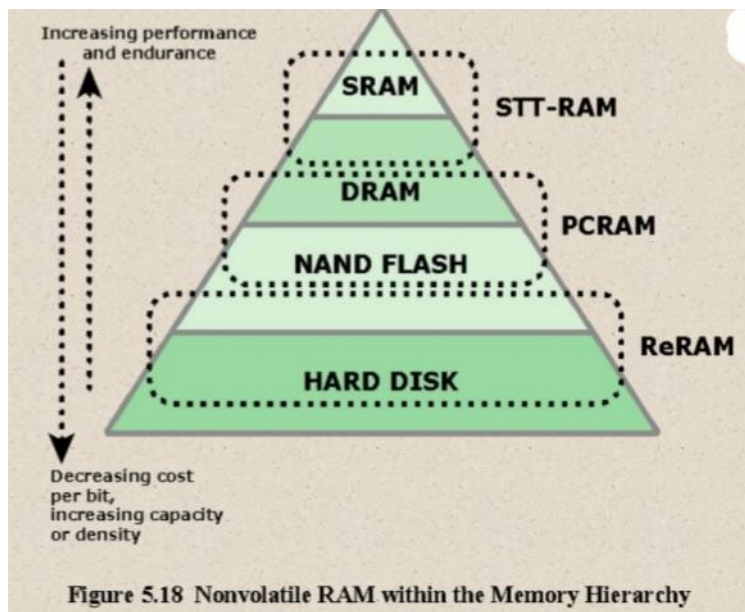
07*DDR Characteristics:

The prefetch buffer is a memory cache located on the SDRAM chip. It enables the SDRAM chip to preposition bits to be placed on the data bus as rapidly as possible.

The DDR I/O bus uses the same clock rate as the memory chip, but because it can handle two bits per cycle, it achieves a data rate that is double the clock rate. The 2-bit prefetch buffer enables the SDRAM chip to keep up with the I/O bus.

08*Flash Memory:

- Used both for internal memory and external memory applications
- First introduced in the mid-1980's
- Is intermediate between EPROM and EEPROM in both cost and functionality
- Uses an electrical erasing technology like EEPROM
- It is possible to erase just blocks of memory rather than an entire chip
- Gets its name because the microchip is organized so that a section of memory cells are erased in a single action
- Does not provide byte-level erasure
- Uses only one transistor per bit so it achieves the high density of EPROM



Memory Type	Category	Erasure	Write Mechanism	Volatility
Random-access memory (RAM)	Read-write memory	Electrically, byte-level	Electrically	Volatile
Read-only memory (ROM)	Read-only memory	Not possible	Masks	Nonvolatile
Programmable ROM (PROM)				
Erased PROM (EPROM)	Read-mostly memory	UV light, chip-level	Electrically	
Electrically Erasable PROM (EEPROM)		Electrically, byte-level		
Flash memory		Electrically, block-level		

Table 5.1
Semiconductor Memory Types

Chapter 06 : Input/Output

01*External Devices:

- Provide a means of exchanging data between the external environment and the computer
- Peripheral device
- An external device connected to an I/O module

02*Three categories:

- Human readable:
 - .Suitable for communicating with the computer user
 - .Video display terminals (VDTs), printers
- Machine readable:
 - .Suitable for communicating with equipment
 - .Magnetic disk and tape systems, sensors and actuators
- Communication:
 - .Suitable for communicating with remote devices such as a terminal, a machine readable device, or another computer

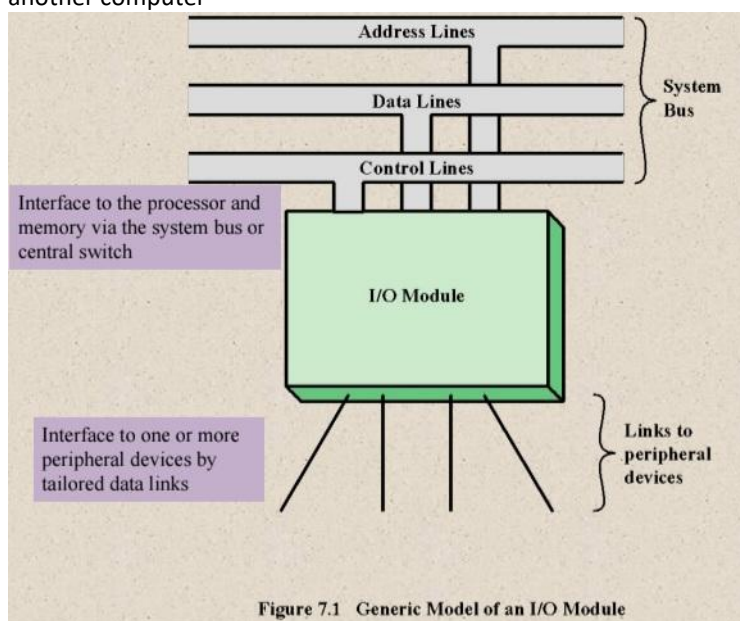


Figure 7.1 Generic Model of an I/O Module

- Control signals: function that the device will perform (INPUT or READ, OUTPUT or WRITE, position a disk head).
- Data: set of bits to be sent to or received from the I/O module.
- Status signals: state of the device (READY/NOT-READY for data transfer).
- Control logic: controls the device's operation in response to direction from the I/O module.
- Transducer: converts data from electrical to other forms of energy during output and from other forms to electrical during input.
- Buffer: temporarily hold data being transferred between the I/O module and the external environment

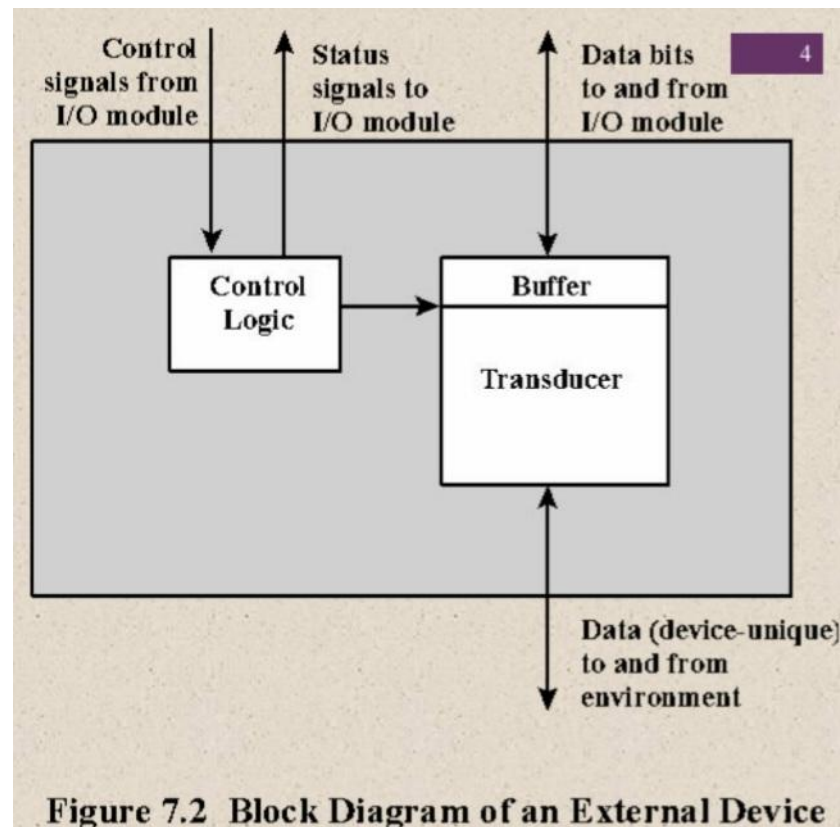


Figure 7.2 Block Diagram of an External Device

03*Keyboard/Monitor:

International Reference Alphabet (IRA):

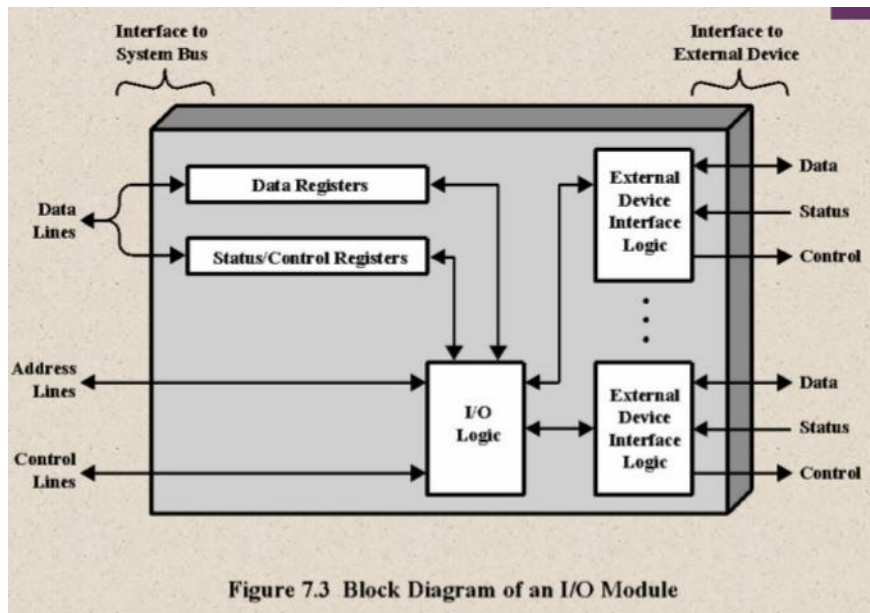
- Basic unit of exchange is the character
- Associated with each character is a code
- Each character in this code is represented by a unique 7-bit binary code
- 128 different characters can be represented

Characters are of two types:

- Printable: Alphabetic, numeric, and special characters
- Control: Have to do with controlling the printing or displaying of characters (e.g. carriage return)

Keyboard Codes:

- When the user depresses a key it generates an electronic signal that is interpreted by the transducer in the keyboard and translated into the bit pattern of the corresponding IRA code
- This bit pattern is transmitted to the I/O module in the computer
- On output, IRA code characters are transmitted to an external device from the I/O module
- The transducer interprets the code and sends the required electronic signals to the output device either to display the indicated character or perform the requested control function



04*Programmed I/O:

Three techniques are possible for I/O operations:

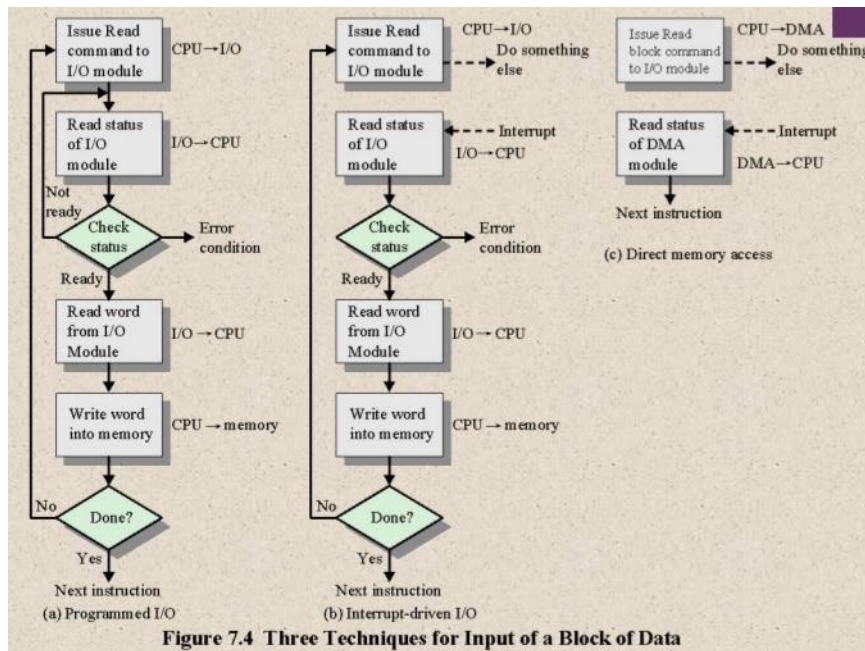
- Programmed I/O
- Data are exchanged between the processor and the I/O module
- Processor executes a program that gives it direct control of the I/O operation
- When the processor issues a command it must wait until the I/O operation is complete
- If the processor is faster than the I/O module this is wasteful of processor time

05*Interrupt-driven I/O:

- Processor issues an I/O command, continues to execute other instructions, and is interrupted by the I/O module when the latter has completed its work
- Direct memory access (DMA)
- The I/O module and main memory exchange data directly without processor involvement

Table 6.1 I/O Techniques

	No Interrupts	Use of Interrupts
I/O-to-memory transfer through processor	Programmed I/O	Interrupt-driven I/O
Direct I/O-to-memory transfer		Direct memory access (DMA)



06*I/O Instructions:

With programmed I/O there is a close correspondence between the I/O-related instructions that the processor fetches from memory and the I/O commands that the processor issues to an I/O module to execute the instructions

The form of the instruction depends on the way in which external devices are addressed

Each I/O device connected through I/O modules is given a unique identifier or address

When the processor issues an I/O

command, the command contains the address of the desired device

Thus each I/O module must interpret the address lines to determine if the command is for itself

Memory-mapped I/O:

There is a single address space for memory locations and I/O devices

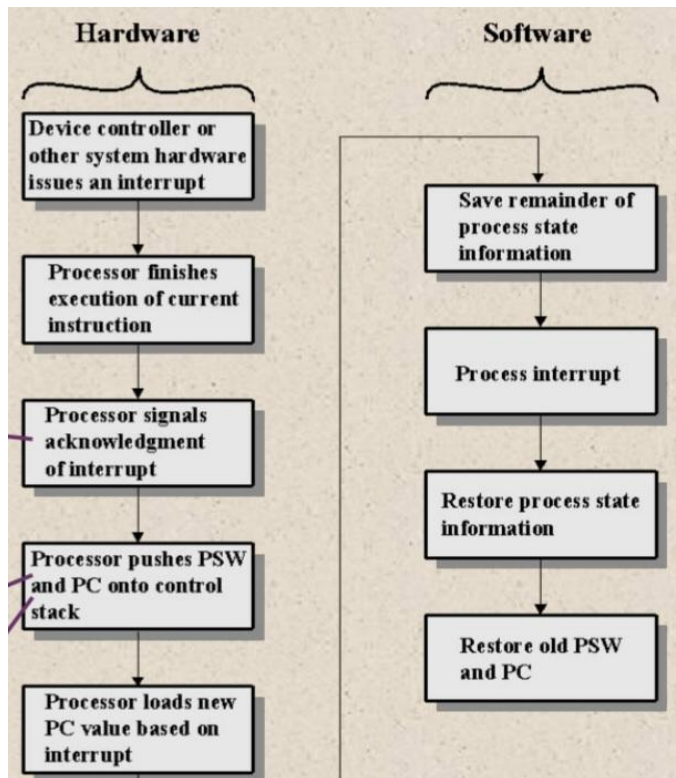
A single read line for memory and a single write line for I/O are needed on the bus

Interrupt-Driven I/O:

The problem with programmed I/O is that the processor has to wait a long time for the I/O module to be ready for either reception or transmission of data

An alternative is for the processor to issue an I/O command to a module and then go on to do some other useful work

The I/O module will then interrupt the processor to request service when it is ready to exchange data with the processor. The processor executes the data transfer and resumes its former processing



07*Design Issues:

- Two design issues arise in implementing interrupt I/O:
- Because there will be multiple I/O modules how does the processor determine which device issued the interrupt?
- If multiple interrupts have occurred how does the processor decide which one to process?

08*Device Identification:

- .Multiple interrupt lines
- Between the processor and the I/O modules
- Most straightforward approach to the problem
- Even if multiple lines are used, it is likely that each line will have multiple I/O modules attached to it

09*Software poll:

- When processor detects an interrupt it branches to an interrupt-service routine whose job is to poll each I/O module to determine which module caused the interrupt
- Time consuming

10*Daisy chain (hardware poll, vectored):

- The interrupt acknowledge line is daisy chained through the modules
- Vector – address of the I/O module or some other unique identifier
- Vectored interrupt – processor uses the vector as a pointer to the appropriate device-service routine, avoiding the need to execute a general interrupt-service routine first

11*Bus arbitration (vectored):

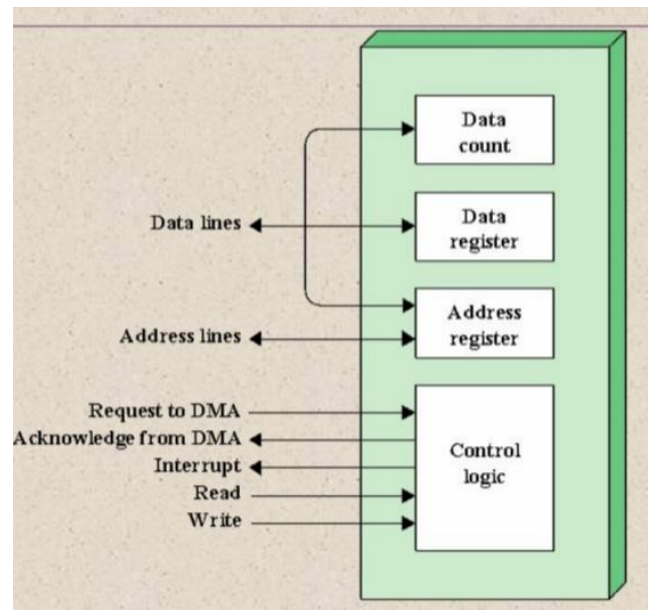
- An I/O module must first gain control of the bus before it can raise the interrupt request line
- When the processor detects the interrupt it responds on the interrupt acknowledge line
- Then the requesting module places its vector on the data lines

12*Drawbacks of Programmed and Interrupt-Driven I/O:

Both forms of I/O suffer from two inherent drawbacks:

- 1)The I/O transfer rate is limited by the speed with which the processor can test and service a device

- 2) The processor is tied up in managing an I/O transfer; a number of instructions must be executed for each I/O transfer
- When large volumes of data are to be moved a more efficient technique is direct memory access (DMA)



13*THE EXTERNAL INTERFACE: FireWire

Serial Bus

- Was developed as an alternative to small computer system interface (SCSI) to be used on smaller systems, such as personal computers, workstations, and servers
- Objective was to meet the increasing demands for high I/O rates while avoiding the bulky and expensive I/O channel technologies developed for mainframe and supercomputer systems
- IEEE standard 1394, for a High Performance Serial Bus
- Uses a daisy chain configuration, with up to 63 devices connected off a single port
- 1022 FireWire buses can be interconnected using bridges
- Provides for hot plugging which makes it possible to connect and disconnect peripherals without having to power the computer system down or reconfigure the system
- Provides for automatic configuration
- No terminations and the system automatically performs a configuration function to assign addresses

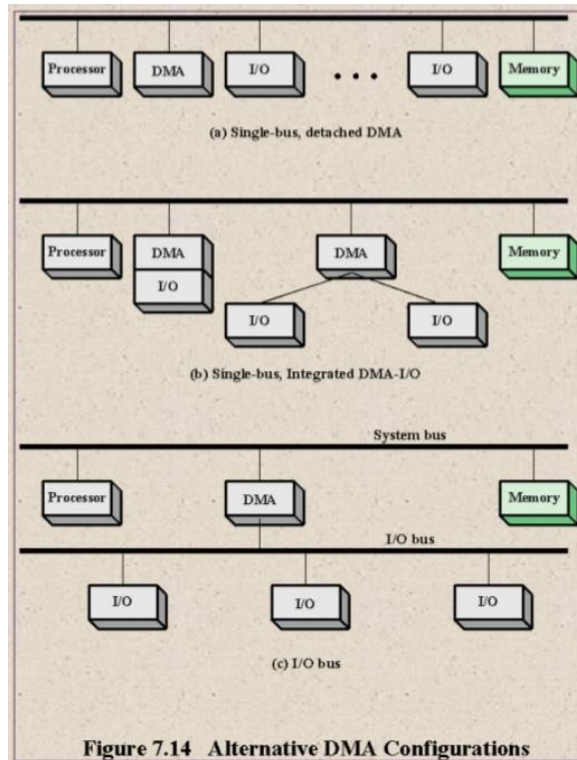
14*THE EXTERNAL INTERFACE:

InfiniBand

- I/O specification aimed at the high-end server market
- First version was released in early 2001
- Heavily relied on by IBM zEnterprise series of mainframes
- Standard describes an architecture and specifications for data flow among processors and intelligent

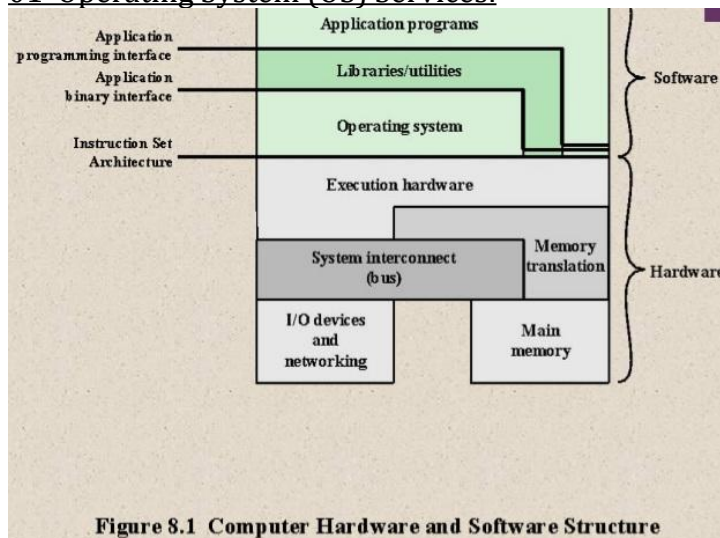
I/O devices

- Has become a popular interface for storage area networking and other large storage configurations
- Enables servers, remote storage, and other network devices to be attached in a central fabric of switches and links
- The switch-based architecture can connect up to 64,000 servers, storage systems, and networking devices



Chapter 07: Operating System Support

01*Operating System (OS) Services:



- The most important system program
- Masks the details of the hardware from the programmer and provides the programmer with a convenient interface for using the system
- The OS typically provides services in the following areas:
 - Program creation
 - Program execution
 - Access to I/O devices
 - Controlled access to files
 - System access
 - Error detection and response
 - Accounting

02*Operating System as Resource Manager:

A computer is a set of resources for the movement, storage, and processing of data and for the control of these functions

- The OS is responsible for managing these resources

The OS as a control mechanism is unusual in two respects:

- The OS functions in the same way as ordinary computer software – it is a program executed by the processor

- The OS frequently relinquishes control and must depend on the processor to allow it to regain control

03*Types of Operating Systems:

Interactive system:

- The user/programmer interacts directly with the computer to request the execution of a job or to perform a transaction

- User may, depending on the nature of the application, communicate with the computer during the execution of the job

Batch system:

- Opposite of interactive

- The user's program is batched together with programs from other users and submitted by a computer operator

- After the program is completed results are printed out for the user

Early Systems:

- From the late 1940s to the mid-1950s the programmer interacted directly with the computer hardware – there was no OS

- Processors were run from a console consisting of display lights, toggle switches, some form of input device and a printer

Problems:

- Scheduling

- Sign-up sheets were used to reserve processor time

- This could result in wasted computer idle time if the user finished early

- If problems occurred the user could be forced to stop before resolving the problem

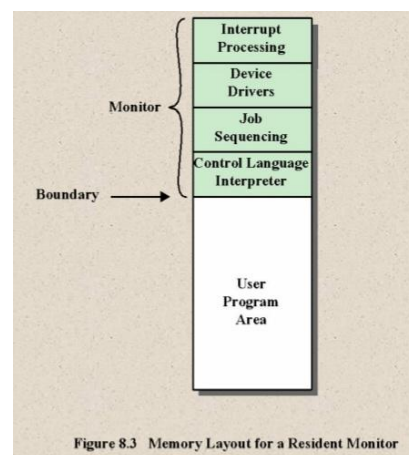
Setup time:

- A single program could involve

- Loading the compiler plus the source program into memory

- Saving the compiled program

- Loading and linking together the object program and common functions



Time Sharing Systems:

- Used when the user interacts directly with the computer
- Processor's time is shared among multiple users
- Multiple users simultaneously access the system through terminals, with the OS interleaving the execution of each user program in a short burst or quantum of computation

Example:

-If there are n users actively requesting service at one time, each user will only see on the average $1/n$ of the effective computer speed

Table 8.3
Batch Multiprogramming versus Time Sharing

	Batch Multiprogramming	Time Sharing
Principal objective	Maximize processor use	Minimize response time
Source of directives to operating system	Job control language commands provided with the job	Commands entered at the terminal

Table 8.4 Types of Scheduling

Long-term scheduling	The decision to add to the pool of processes to be executed
Medium-term scheduling	The decision to add to the number of processes that are partially or fully in main memory
Short-term scheduling	The decision as to which available process will be executed by the processor
I/O scheduling	The decision as to which process's pending I/O request shall be handled by an available I/O device

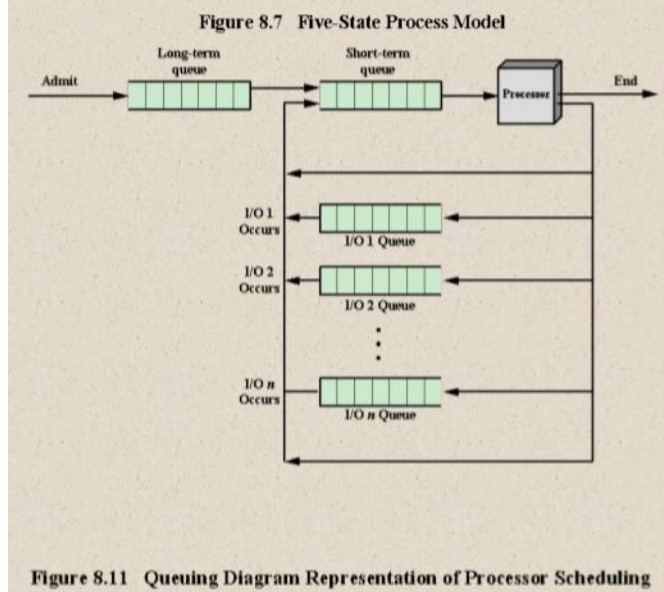
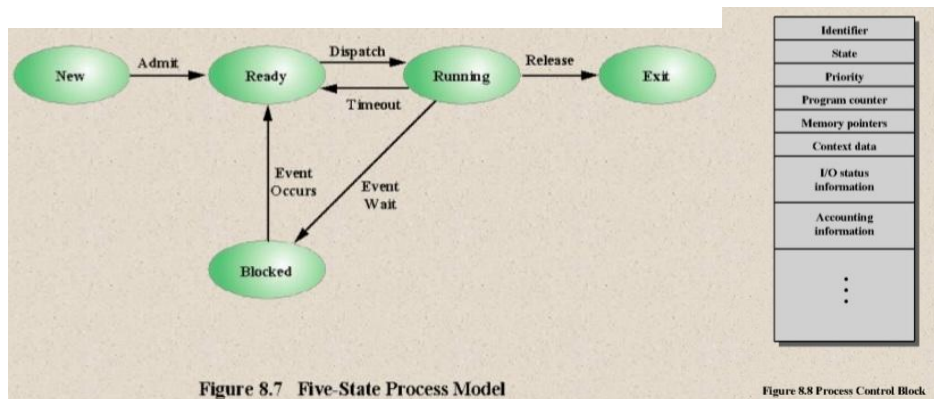
04*Medium-Term Scheduling and Short-Term Scheduling:

Medium Term

- Part of the swapping function
- Swapping-in decision is based on the need to manage the degree of multiprogramming
- Swapping-in decision will consider the memory requirements of the swapped-out processes

Short-Term:

- Also known as the dispatcher
- Executes frequently and makes the fine-grained decision of which job to execute next



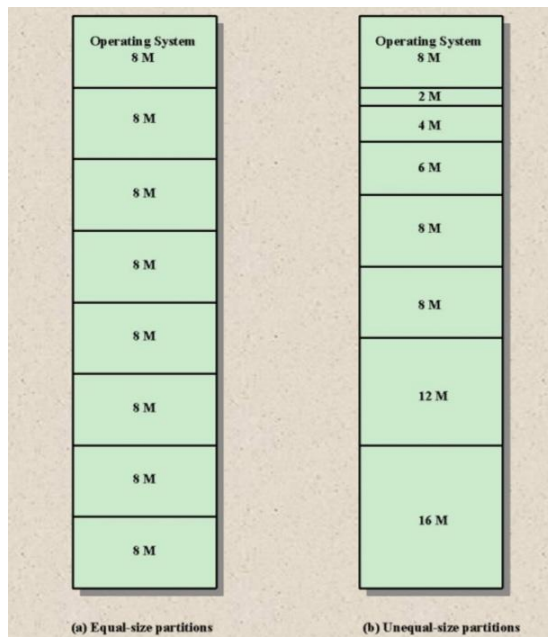


Figure 8.13 Example of Fixed Partitioning of a 64-Mbyte Memory

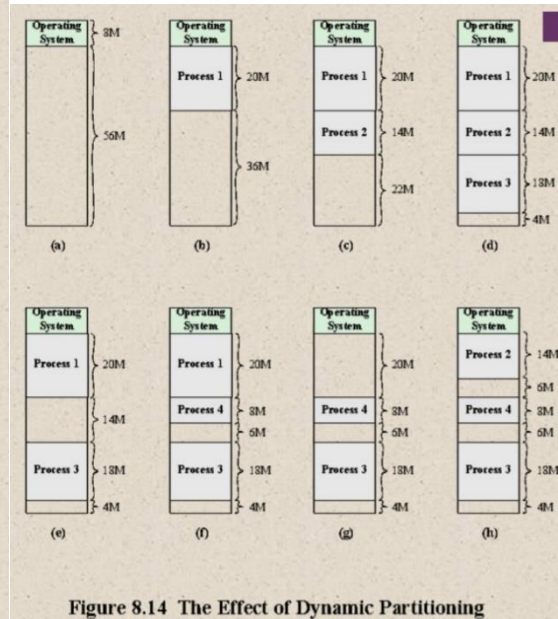


Figure 8.14 The Effect of Dynamic Partitioning

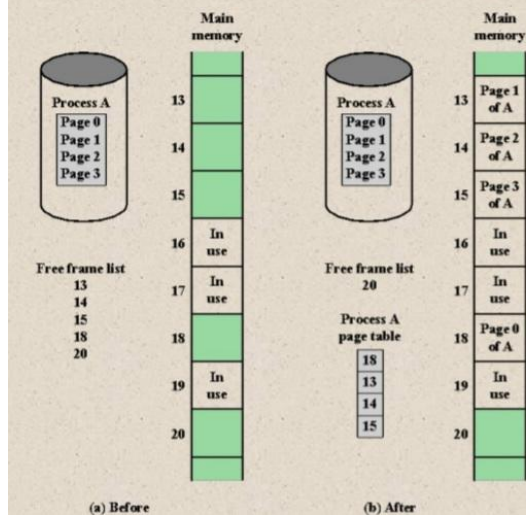


Figure 8.15 Allocation of Free Frames

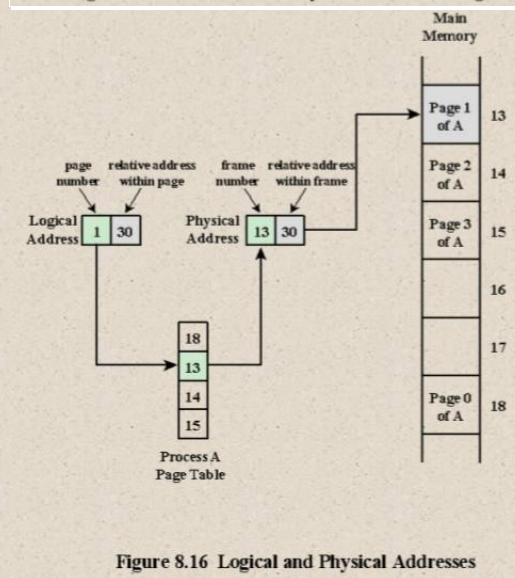


Figure 8.16 Logical and Physical Addresses

Virtual Memory

- Each page of a process is brought in only when it is needed
- Principle of locality
- When working with a large process execution may be confined to a small section of a program (subroutine)
- It is better use of memory to load in just a few pages
- If the program references data or branches to an instruction on a page not in main memory, a page fault is triggered which tells the OS to bring in the desired page

Advantages:

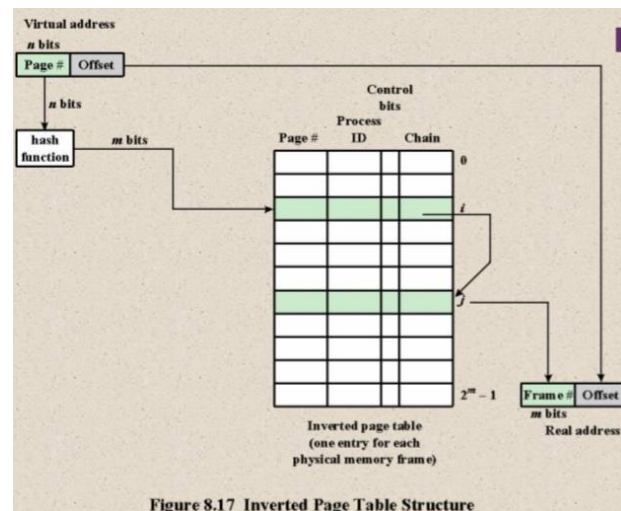
- More processes can be maintained in memory
- Time is saved because unused pages are not swapped in and out of memory

Disadvantages:

- When one page is brought in, another page must be thrown out (page replacement)
- If a page is thrown out just before it is about to be used the OS will have to go get the page again

Thrashing:

When the processor spends most of its time swapping pages rather than executing instructions



Chapter 08 : Instruction Sets: Addressing

01*Addressing Modes Immediate:

Direct

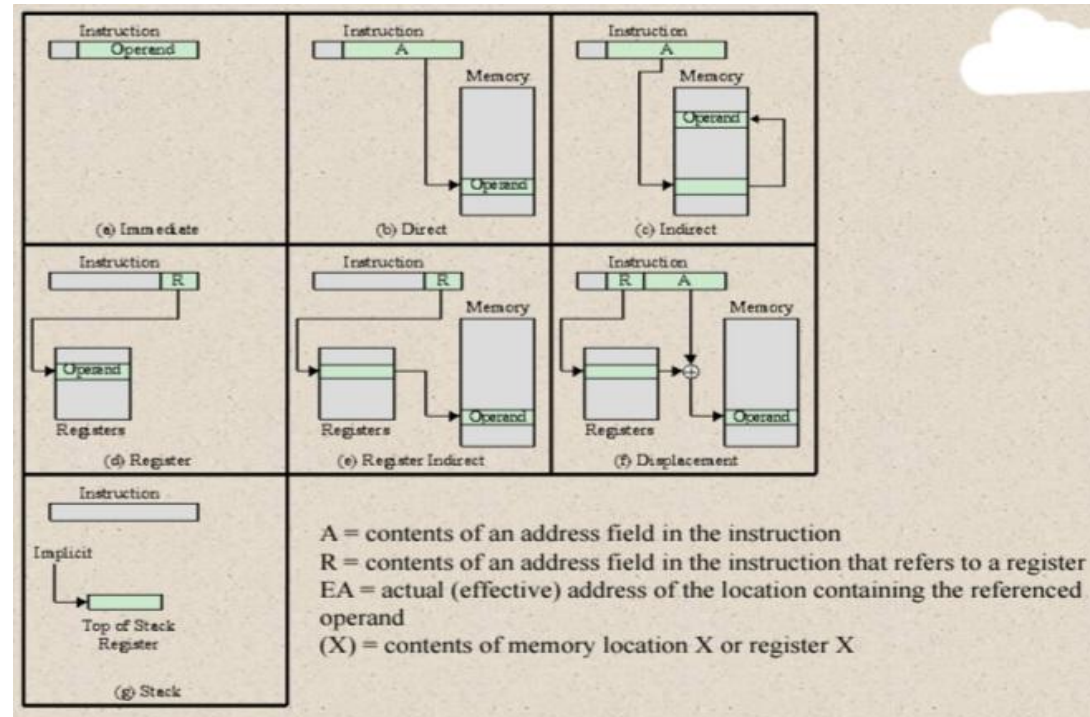
Indirect

Register

Register indirect

Displacement

Stack



Immediate Addressing:

-Simplest form of addressing

-Operand = A

-This mode can be used to define and use constants or set initial values of variables

- Typically the number will be stored in twos complement form
- The leftmost bit of the operand field is used as a sign bit

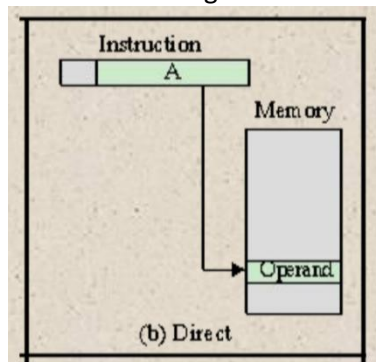
Advantage:

No memory reference other than the instruction fetch is required to obtain the operand, thus saving one memory or cache cycle in the instruction cycle

Disadvantage:

-The size of the number is restricted to the size of the address field, which, in most instruction sets, is small compared with the word length

Direct Addressing:



Indirect Addressing:

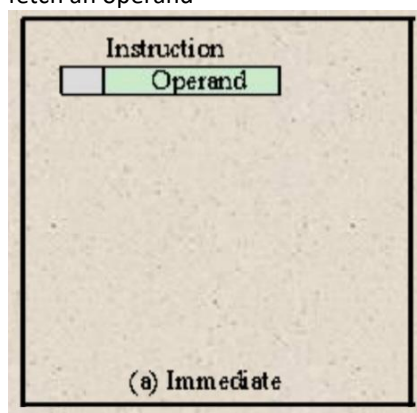
- Reference to the address of a word in memory which contains a full-length address of the operand
- $EA = (A)$
- Parentheses are to be interpreted as meaning contents of

Advantage:

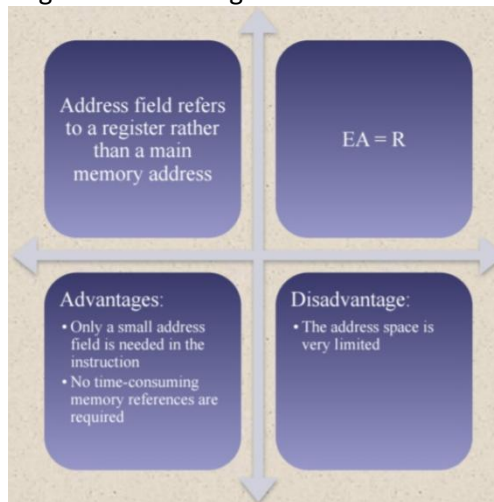
- For a word length of N an address space of $2N$ is now available

Disadvantage:

- Instruction execution requires two memory references to fetch the operand
- One to get its address and a second to get its value
- A rarely used variant of indirect addressing is multilevel or cascaded indirect addressing
- $EA = (\dots (A) \dots)$
- Disadvantage is that three or more memory references could be required to fetch an operand



Register Addressing:



Register Indirect Addressing:

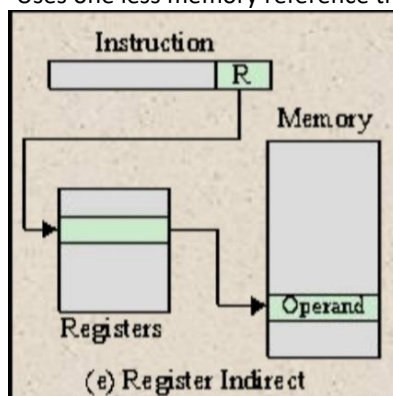
-Analogous to indirect addressing

-The only difference is whether the address field refers to a memory location or a register

- $EA = (R)$

-Address space limitation of the address field is overcome by having that field refer to a word-length location containing an address

-Uses one less memory reference than indirect addressing



Displacement Addressing:

-Combines the capabilities of direct addressing and register indirect addressing

- $EA = A + (R)$

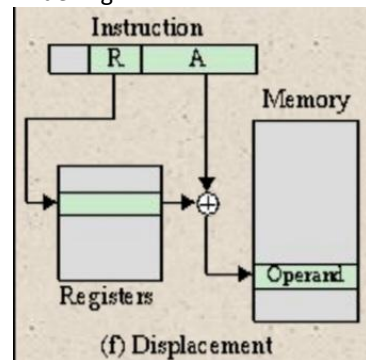
-Requires that the instruction have two address fields, at least one of which is explicit

-The value contained in one address field (value = A) is used directly

The other address field refers to a register whose contents are added to A to produce the effective address

Most common uses:

- Relative addressing
- Base-register addressing
- Indexing



Relative Addressing:

The implicitly referenced register is the program counter (PC)

- The next instruction address is added to the address field to produce the EA
- Typically the address field is treated as a two's complement number for this operation
- Thus the effective address is a displacement relative to the address of the instruction

Exploits the concept of locality:

- Saves address bits in the instruction if most memory

Base-Register Addressing:

- The referenced register contains a main memory address and the address field contains displacement from that address
- The register reference may be explicit or implicit
- Exploits the locality of memory references

Indexing:

- The address field references a main memory address and the referenced register contains a positive displacement from that address
- The method of calculating the EA is the same as for base-register addressing
- An important use is to provide an efficient mechanism for performing iterative operations

Autoindexing:

- Automatically increment or decrement the index register after each reference to it
- $EA = A + (R)$
- (R) $\rightarrow (R) + 1$

Postindexing:

- Indexing is performed after the indirection
- $EA = (A) + (R)$

Preindexing:

- Indexing is performed before the indirection
- $EA = (A + (R))$

Stack Addressing:

- A stack is a linear array of locations
- Sometimes referred to as a pushdown list or last-in-first-out queue
- A stack is a reserved block of locations
- Items are appended to the top of the stack so that the block is partially filled
- Associated with the stack is a pointer whose value is the address of the top of the stack

- The stack pointer is maintained in a register
- Thus references to stack locations in memory are in fact register indirect addresses
- Is a form of implied addressing
- The machine instructions need not include a memory reference but implicitly operate on the top of the stack

Mode	Algorithm	Principal Advantage	Principal Disadvantage
Immediate	Operand = A	No memory reference	Limited operand magnitude
Direct	EA = A	Simple	Limited address space
Indirect	EA = (A)	Large address space	Multiple memory references
Register	EA = R	No memory reference	Limited address space
Register indirect	EA = (R)	Large address space	Extra memory reference
Displacement	EA = A + (R)	Flexibility	Complexity
Stack	EA = top of stack	No memory reference	Limited applicability

A = contents of an address field in the instruction
R = contents of an address field in the instruction that refers to a register
EA = actual (effective) address of the location containing the referenced operand
(X) = contents of memory location X or register X

ARM Data Processing Instruction Addressing and Branch Instructions:

- Data processing instructions
- Use either register addressing or a mixture of register and immediate addressing
- For register addressing the value in one of the register operands may be scaled using one of the five shift operators

Branch instructions:

- The only form of addressing for branch instructions is immediate
- Instruction contains 24 bit value
- Shifted 2 bits left so that the address is on a word boundary
- Effective range $\pm 32\text{MB}$ from from the program counter

Instruction Formats:

Define the layout of the bits of an instruction, in terms of its constituent fields Must include an opcode and, implicitly or explicitly, indicate the addressing mode for each operand
For most instruction sets more than one instruction format is used

Instruction Length:

- Most basic design issue
- Affects, and is affected by:
- Memory size
- Memory organization
- Bus structure
- Processor complexity

Processor speed:

- Should be equal to the memory-transfer length or one should be a multiple of the other
- Should be a multiple of the character length, which is usually 8 bits, and of the length of fixed-point numbers

Allocation of Bits:

Number of addressing modes	Number of operands	Register versus memory
Number of register sets	Address range	Address granularity

Variable-Length Instructions:

- Variations can be provided efficiently and compactly
- Increases the complexity of the processor
- Does not remove the desirability of making all of the instruction lengths integrally related to word length
- Because the processor does not know the length of the next instruction to be fetched a typical strategy is to fetch a number of bytes or words equal to at least the longest possible instruction
- Sometimes multiple instructions are fetched

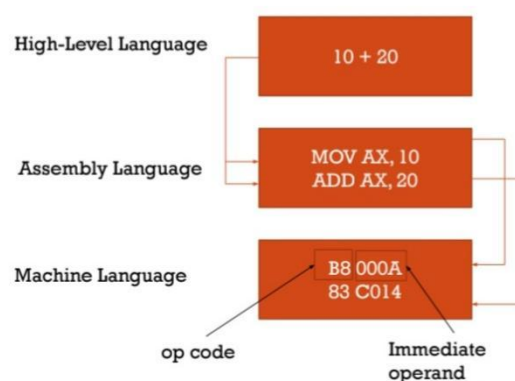
Chapter 09 : Assembly

01*A HIGH-LEVEL LANGUAGE:

- programming language that enables development of a program in a much more user-friendly programming context
- generally independent of the computer's hardware architecture.
- has a higher level of abstraction from the computer,
- focuses more on the programming logic rather than the underlying hardware components such as memory addressing and register utilization.

02*WHAT IS ASSEMBLY LANGUAGE?

- Each personal computer has a microprocessor that manages the computer's arithmetical, logical and control activities.
- Each family of processors has its own set of instructions: machine language instruction
- getting input from keyboard,
- displaying information on screen
- performing various other jobs.
- Processor understands only machine language instructions which are strings of 1s and 0s.
- too obscure and complex for using in software development.
- The low level assembly language is designed for a specific family of processors that represents various instructions in symbolic code and a more understandable form.
- High-Level Language (HLL) code gets translated into machine language.
- HLL can be C, C++, C#, JAVA, VB,
- Machine language: The language a particular processor understands
- Assembly language: machine-specific language with a one-to-one correspondence with the machine language for that computer



03*WHY STUDY ASSEMBLY LANGUAGE (ASM)?

- To learn the computer (i.e., architecture, hardware, operating system...)
- by direct access to memory, video controller, sound card, keyboard...
- by communicating with the operating system
- Understand what's happening "under the hood"
- Someone needs to develop future computer systems
- To speed up applications
- direct access to hardware (ex: writing directly to I/O ports instead of doing a system call)
- good ASM code is faster and smaller

- Avoid restrictions of high level languages

04*PROBLEMS WITH ASSEMBLY LANGUAGE:

- Provides no structure
- Is not portable
- Applications can be very long
- “Hard” to read and understand
- Lots of detail required

05*ASSEMBLY REGISTERS:

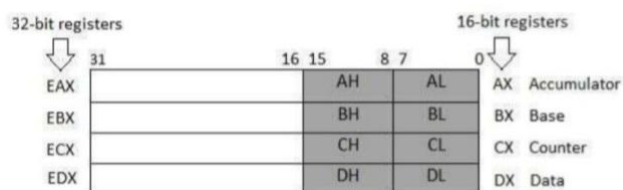
- Processor operations mostly involve processing data.
- This data can be stored in memory and accessed from thereon.
- To speed up the processor operations, the processor includes some internal memory storage locations, called registers.
- The registers stores data elements for processing without having to access the memory.
- A limited number of registers are built into the processor chip.

06*PROCESSOR REGISTERS:

- There are ten 32-bit and six 16-bit processor registers in IA-32 architecture.
- The registers are grouped into three categories:
- General registers
- Data registers
- Pointer registers
- Index registers
- Control registers
- Segment registers

07*DATA REGISTERS:

- Four 32-bit data registers are used for arithmetic, logical and other operations.
- AX: used in input/output and most arithmetic instructions (primary accumulator)
 BX: used in indexed addressing.
 CX: stores the loop count in iterative operations.
 DX: used with AX register along with DX for multiply and divide operations involving large values



08*POINTER REGISTERS:

- The pointer registers are 32-bit EIP, ESP and EBP registers and corresponding 16-bit right portions IP, SP and BP.
- Instruction Pointer (IP) –
- stores the offset address of the next instruction to be executed.
- Value changed:
- Automatically to implement sequential control flow
- By jump instructions to implement selection, repetition
- Stack Pointer (SP) –
- provides the offset value within the program stack.
- Base Pointer (BP)
- helps in referencing the parameter variables passed to a subroutine.

09*INDEX REGISTERS:

- The 32-bit index registers ESI and EDI and their 16-bit rightmost portions SI and DI are used for indexed addressing and sometimes used in addition and subtraction.
- There are two sets of index pointers:
- Source Index (SI) - it is used as source index for string operations
- Destination Index (DI) - it is used as destination index for string operations

10*CONTROL REGISTERS:

- Many instructions involve comparisons and mathematical calculations and change the status of the flags and some other conditional instructions test the value of these status flags to take the control flow to other location.
- each flag is a single binary bit
- A flag is set when it equals 1; it is clear (or reset) when it equals 0

11*COMMON FLAGS BIT:

Interrupt Flag (IF):

.Determines whether the external interrupts like, keyboard entry etc. are to be ignored or processed. It disables the external interrupt when the value is 0 and enables interrupts when set to 1.

Sign Flag (SF):

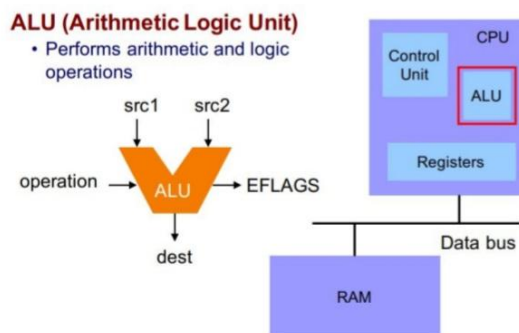
.A positive result clears the value of SF to 0 and negative result sets it to 1.

Zero Flag (ZF):

.indicates the result of an arithmetic or comparison operation. A nonzero result clears the zero flag to 0, and a zero result sets it to 1.

Parity Flag (PF):

.indicates the total number of 1-bits in the result obtained from an arithmetic operation. An even number of 1-bits clears the parity flag to 0 and an odd number of 1-bits sets the parity flag to 1.



12*SEGMENT REGISTERS:

▪Segments are specific areas defined in a program for containing data, code and stack. There are three main segments:

▪Code Segment:

contains all the instructions to be executed.

▪CS register stores the starting address of the code segment.

▪Data Segment:

contains data, constants and work areas.

DS register stores the starting address of the data segment.

▪Stack Segment:

contains data and return addresses of procedures or subroutines. It is implemented as a 'stack' data structure.

The Stack Segment register or SS register stores the starting address of the stack.

The segment registers stores the starting addresses of a segment.
 To get the exact location of data or instruction within a segment, an offset value (or displacement) is required. To reference any memory location in a segment, the processor combines the segment address in the segment register with the offset value of the location.
 IP in association with the CS register (as CS:IP) gives the complete address of the current instruction in the code segment.

13*SECTION OF ASSEMBLY PROGRAM:

```
section .data
section .bss
section .text
    gloable main
main:
```

14*THE DATA SECTION:

- The data section is used for declaring initialized data or constants.
- This data does not change at runtime.
- You can declare various constant values, file names or buffer size etc. in this section.

```
section .data
section .bss
section .text
    gloabl main
main:
```

15*THE BSS SECTION:

- The bss section is used for declaring variables. The text section is used for keeping the actual code.
- This section must begin with the declaration global main, which tells the kernel where the program execution begins.

```
section .data
section .bss
section .text
    gloabl main
main:
```

16*COMMENTS:

- Assembly language comment begins with a semicolon (;).
- It may contain any printable character including blank.
- It can appear on a line by itself, like:
- or, on the same line along with an instruction, like:

```
add eax , ebx ; adds ebx to eax
```

17*SYNTAX OF ASSEMBLY LANGUAGE STATEMENTS:

- Assembly language statements are entered one statement per line.
- Each statement follows the following format:
- The fields in the square brackets are optional.

```
[Label] mnemonic [operands] [;comments]
```

EXAMPLES:

```
INC COUNT      ; Increment the memory variable COUNT
MOV TOTAL, 48   ; Transfer the value 48 in the
                ; memory variable TOTAL
ADD AH, BH      ; Add the content of the
                ; BH register into the AH register
AND MASK1, 128  ; Perform AND operation on the
                ; variable MASK1 and 128
ADD MARKS, 10   ; Add 10 to the variable MARKS
MOV AL, 10      ; Transfer the value 10 to the AL register
```

18*LINUX SYSTEM CALLS:

- You need to take the following steps for using Linux system calls in your program:
- Put the system call number in the EAX register (primary accumulator)
- Store the arguments to the system call in the registers EBX (base register), ECX (count register), etc.
- Call the relevant interrupt (80h)
- The result is usually returned in the EAX register

19*SOME OF THE SYSTEM CALLS:

%eax	Name	%ebx	%ecx	%edx	%esx	%edi
1	sys_exit	int	-	-	-	-
2	sys_fork	struct pt_regs	-	-	-	-
3	sys_read	unsigned int	char *	size_t	-	-
4	sys_write	unsigned int	const char *	size_t	-	-
5	sys_open	const char *	int	int	-	-
6	sys_close	unsigned int	-	-	-	-

20*LINUX SYSTEM CALLS:

- The following code snippet shows the use of the system call sys_exit:

```
mov    eax,1
int    0x80
```

- The following code snippet shows the use of the system call sys_write:

```
mov    edx,4
mov    ecx,msg
mov    ebx,1
mov    eax,4
int    0x80
```

21*ALLOCATING STORAGE SPACE FOR INITIALIZED DATA:

- The syntax for storage allocation statement for initialized data is:

[variable-name] define-directive initial-value [,initial-value]...

Where variable-name is the identifier for each storage space. The assembler associates an offset value for each variable name defined in the data segment. There are five basic forms of the define directive:

Directive	Purpose	Storage Space
DB	Define Byte	allocates 1 byte
DW	Define Word	allocates 2 bytes
DD	Define Doubleword	allocates 4 bytes
DQ	Define Quadword	allocates 8 bytes
DT	Define Ten Bytes	allocates 10 bytes

EXAMPLES:

- You can have multiple data definition statements in a program

```
choice    DB    'Y'                ;ASCII of y = 79H
number1   DW    12345              ;12345D = 3039H
number2   DD    12345679          ;123456789D = 75BCD15H
```

NOTE:

- Each byte of character is stored as its ASCII value in hexadecimal
- Each decimal value is automatically converted to its 16-bit binary equivalent and stored as a hexadecimal number

- Processor uses the little-endian byte ordering (next slide)
- Negative numbers are converted to its 2's complement representation
- Short and long floating-point numbers are represented using 32 or 64 bits, respectively

22*BYTE ORDER:

```

1000 00000101
1001 00000000
1002 00000000
1003 00000000

1000 00000000
1001 00000000
1002 00000000
1003 00000101

```

The following program shows use of the define directive:

```

section .text
    global main      ;must be declared for linker (gcc)
main:               ;tell linker entry point

    mov     edx,1     ;message length
    mov     ecx,choice ;message to write
    mov     ebx,1     ;file descriptor (stdout)
    mov     eax,4     ;system call number (sys_write)
    int     0x80      ;call kernel

    mov     eax,1     ;system call number (sys_exit)
    int     0x80      ;call kernel

section .data
choice DB 'y'

```

When the above code is compiled and executed, it produces following result:

```
y
```

23*MULTIPLE INITIALIZATIONS:

- The TIMES directive allows multiple initializations to the same value. marks Times 9 DM 0
- For example, an array named marks of size 9 can be defined and initialized to zero using the following statement:
- The TIMES directive is useful in defining arrays

THE FOLLOWING PROGRAM DISPLAYS 9 ASTERISKS ON THE SCREEN:

```

section .text
    global main      ;must be declared for linker (ld)
main:               ;tell linker entry point

    mov     edx,9     ;message length
    mov     ecx,stars  ;message to write
    mov     ebx,1     ;file descriptor (stdout)
    mov     eax,4     ;system call number (sys_write)
    int     0x80      ;call kernel

    mov     eax,1     ;system call number (sys_exit)
    int     0x80      ;call kernel

section .data
stars     times 9 db '*'

```

When the above code is compiled and executed, it produces following result:

```
*****
```

24*THE EQU DIRECTIVE:

- Syntax:
CONSTANT_NAME EQU expression
- For example,
TOTAL_STUDENTS equ 50

You can then use this constant value in your code, like:

```

mov ecx, TOTAL_STUDENTS
cmp eax, TOTAL_STUDENTS

```

- The operand of an EQU statement can be an expression:

```

LENGTH equ 20
WIDTH equ 10
AREA equ length * width

```

- Above code segment would define AREA as 200.

25*THE %ASSIGN DIRECTIVE:

- The %assign directive can be used to define numeric constants like the EQU directive.
- This directive allows redefinition.

For example,

- you may define the constant TOTAL as: %assign TOTAL 10
- Later in the code you can redefine it as: %assign TOTAL 20
- This directive is case-sensitive.

26*THE %DEFINE DIRECTIVE:

- The %define directive allows defining both numeric and string constants.
- This directive is similar to the #define in C.
- For example, you may define the constant PTR as: %define PTR [EBP+4]
- The above code replaces PTR by [EBP+4].
- This directive also allows redefinition and it is case sensitive.

27*THE INC INSTRUCTION:

- The INC instruction is used for incrementing an operand by one.
- It works on a single operand that can be either in a register or in memory.

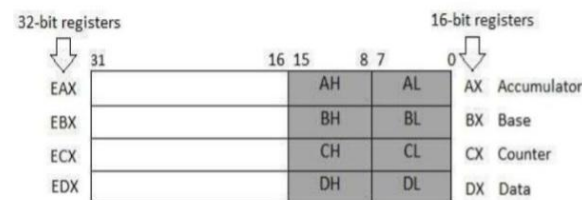
▪SYNTAX:

INC destination

- The operand destination could be an 8-bit, 16-bit or 32-bit operand.

▪EXAMPLE:

```
INC EBX ; Increments 32-bit register
INC DL ; Increments 8-bit register
INC [count] ; Increments the count variable
```



28*THE DEC INSTRUCTION:

- The DEC instruction is used for decrementing an operand by one.
- It works on a single operand that can be either in a register or in memory.

▪SYNTAX:

DEC destination

- The operand destination could be an 8-bit, 16-bit or 32-bit operand.

```
.data
myWord WORD 1000h
myDword DWORD 10000000h
.code
inc myWord ; 1001h
dec myWord ; 1000h
inc myDword ; 10000001h
mov ax,00FFh
inc ax ; AX = 0100h
mov ax,00FFh
inc al ; AX = 0000h
```

29*INSTRUCTIONS:

▪The ADD and SUB instructions are used for performing simple addition/subtraction of binary data in byte, word and doubleword size, i.e., for adding or subtracting 8-bit, 16-bit or 32-bit operands respectively.

▪SYNTAX:

ADD/SUB destination, source

▪The ADD/SUB instruction can take place between:

▪Register to register

▪Memory to register

▪Register to memory

▪Register to constant data

▪Memory to constant data

▪However, memory-to-memory operations are not possible using ADD/SUB instructions.

▪An ADD or SUB operation sets or clears the overflow and carry flags.

```
MOV eax, 10000h
ADD eax, 40000h
SUB eax, 20000h
```

30*THE MUL/IMUL INSTRUCTION:

▪There are two instructions for multiplying binary data.

▪MUL (Multiply) instruction handles unsigned data

▪IMUL (Integer Multiply) handles signed data.

▪Both instructions affect the Carry and Overflow flag.

▪SYNTAX:

MUL/IMUL multiplier

31*MUL INSTRUCTION:

▪The MUL (unsigned multiply) instruction multiplies an 8-, 16-, or 32-bit operand by either AL, AX, or EAX.

▪Multiplicand will be in an accumulator

▪Generated product is stored in two registers depending upon the size of the operands.

▪The instruction formats are:

▪MUL r/m8 ▪MUL r/m16 ▪MUL r/m32

Implied operands:

Multiplicand	Multiplier	Product
AL	r/m8	AX
AX	r/m16	DX:AX
EAX	r/m32	EDX:EAX

The Carry flag indicates whether or not the upper half of the product contains significant digits.

```
mov eax,12345h
mov ebx,1000h
mul ebx ; EDX:EAX = 0000000012345000h, CF=0
```

32*IMUL INSTRUCTION:

▪IMUL (signed integer multiply) multiplies an 8-, 16-, or 32-bit signed operand by either AL, AX, or EAX

▪Preserves the sign of the product by sign-extending it into the upper half of the destination register

▪The high byte contains a copy of the sign bit from the low byte:

Default Operands:

Dividend	Divisor	Quotient	Remainder
AX	r/m8	AL	AH
DX:AX	r/m16	AX	DX
EDX:EAX	r/m32	EAX	EDX

33*THE DIV/IDIV INSTRUCTIONS:

- The division operation generates two elements: a quotient and a remainder.

- Overflow:

In case of multiplication, overflow does not occur because double-length registers are used to keep the product.

However, in case of division, overflow may occur. The processor generates an interrupt if overflow occurs.

The DIV (Divide) instruction is used for unsigned data and the IDIV (Integer Divide) is used for signed data.

The dividend is in an accumulator

- SYNTAX:

The DIV (unsigned divide) instruction performs 8-bit, 16-bit, and 32-bit division on unsigned integers

A single operand is supplied (register or memory operand), which is assumed to be the divisor

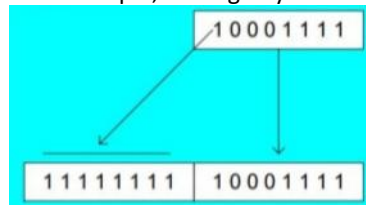
Instruction formats: DIV r/m8 DIV r/m16 DIV r/m32

34*SIGNED INTEGER DIVISION:

- Signed integers must be sign-extended before division takes place

- fill high byte/word/doubleword with a copy of the low byte/word/doubleword's sign bit

- For example, the high byte contains a copy of the sign bit from the low byte:



35*DIV INSTRUCTION:

- DIV (signed divide) performs signed integer division

- Uses same operands as DIV

- Example: 8-bit division of -48 by 5

```
mov al,-48
cbw ; extend AL into AH
mov bl,5
idiv bl ; AL = -9, AH = -3
```

36*IMPLEMENTING ARITHMETIC EXPRESSIONS:

- Example: $\text{var4} = (\text{var1} + \text{var2}) * \text{var3}$

```
mov eax,var1
add eax,var2
mul var3
jo TooBig ; check for overflow
mov var4,eax ; save product
```

37*FLAGS AFFECTED BY ARITHMETIC:

- The ALU has a number of status flags that reflect the outcome of arithmetic (and bitwise) operations

- based on the contents of the destination operand

- Essential flags:

Zero flag – destination equals zero

Sign flag – destination is negative

Carry flag – unsigned value out of range

Overflow flag – signed value out of range

The MOV instruction never affects the flags.

38*ZERO FLAG (ZF):

- Whenever the destination operand equals Zero, the Zero flag is set.

```
mov cx,1
sub cx,1 ; CX = 0, ZF = 1
mov ax,0FFFFh
inc ax ; AX = 0, ZF = 1
inc ax ; AX = 1, ZF = 0
```

A flag is set when it equals 1.

A flag is clear when it equals 0. SIGN FLAG (SF)

- The Sign flag is set when the destination operand is negative. The flag is clear when the destination is positive.

```
mov cx,0
sub cx,1 ; CX = -1, SF = 1
add cx,2 ; CX = 1, SF = 0
```

The sign flag is a copy of the destination's highest bit:

```
mov al,0
sub al,1 ; AL=11111111b, SF=1
add al,2 ; AL=00000001b, SF=0 CARRY FLAG (CF)
```

- Addition and CF: copy carry out of MSB to CF
- Subtraction and CF: copy inverted carry out of MSB to CF
- INC/DEC do not affect CF
- Applying NEG to a nonzero operand sets CF
- For each of the following marked entries, show the values of the destination operand and the Sign, Zero, and Carry flags:

```
mov ax,00FFh
add ax,1 ; AX= SF= ZF= CF=
sub ax,1 ; AX= SF= ZF= CF=
add al,1 ; AL= SF= ZF= CF=
mov bh,6Ch
add bh,95h ; BH= SF= ZF= CF=
mov al,2
sub al,3 ; AL= SF= ZF= CF=
0100h 0 0 0
00FFh 0 0 0
00h 0 1 1
01h 0 0 1
FFh 1 0 1
```

39*OVERFLOW FLAG (OF):

- The Overflow flag is set when the signed result of an operation is invalid or out of range.
- The two examples are identical at the binary level because 7Fh equals +127. To determine the value of the destination operand, it is often easier to calculate in hexadecimal.

```
; Example 1
mov al,+127
add al,1 ; OF = 1, AL = ??
; Example 2
mov al,7Fh ; OF = 1, AL = 80h
add al,1 A RULE OF THUMB
```

- When adding two integers, remember that the Overflow flag is only set when . . .
- Two positive operands are added and their sum is negative
- Two negative operands are added and their sum is positive

40*INTEGERS: HARDWARE

- All CPU instructions operate exactly the same on signed and unsigned integers
- The CPU cannot distinguish between signed and unsigned integers
- YOU, the programmer, are solely responsible for using the correct data type with each instruction

41*OVERFLOW/CARRY FLAGS:

HARDWARE VIEWPOINT

- How the ADD instruction modifies OF and CF:
- CF = (carry out of the MSB)
- OF = (carry out of the MSB) XOR (carry into the MSB)
- How the SUB instruction modifies OF and CF:
- NEG the source and ADD it to the destination
- CF = INVERT (carry out of the MSB)
- OF = (carry out of the MSB) XOR (carry into the MSB)

42*PARITY (PF) FLAG:

- PF is set when LSB of the destination has an even number of 1 bits.

```
mov al, 10001100b
add al, 00000010b ; AL=10001110, PF=1
sub al, 10000000b ; AL=00001110, PF=0
```

43*JMP INSTRUCTION:

- JMP is an unconditional jump to a label that is usually within the same procedure.
- Syntax: JMP target
- Logic: EIP $\leftarrow$ target
- Example:

```
top:
.
.
jmp top LOOP INSTRUCTION
```

- The LOOP instruction creates a counting loop
 - Syntax: LOOP label
 - Where, label is the target label that identifies the target instruction as in the jump instructions.
 - The LOOP instruction assumes that the ECX register contains the loop count.
 - When the loop instruction is executed, the ECX register is decremented and the control jumps to the target label, until the ECX register value, i.e., the counter reaches the value zero
 - Logic: ECX $\leftarrow$ ECX - 1
- if ECX != 0, jump to label

44*NOT INSTRUCTION:

- Performs a bitwise Boolean NOT operation on a single destination operand
- Syntax:
NOT destination
- Example:

```
mov al, 11110000b
not al
```

NOT	
X	$\neg X$
F	T
T	F

NOT	00111011	
	11000100	inverted

45*AND INSTRUCTION:

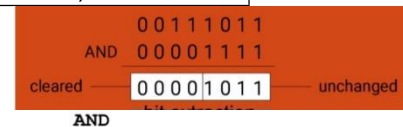
▪Performs a bitwise Boolean AND operation between each pair of matching bits in two operands

▪Syntax:

AND destination, source

▪Example:

```
mov al, 00111011b
and al, 00001111b
```



AND		
x	y	$x \wedge y$
0	0	0
0	1	0
1	0	0
1	1	1

46*OR INSTRUCTION:

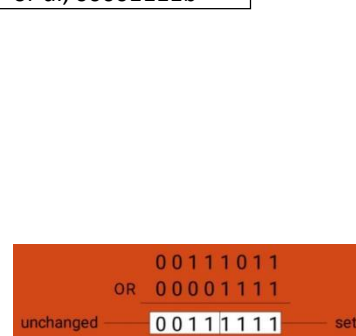
▪Performs a bitwise Boolean OR operation between each pair of matching bits in two operands

▪Syntax:

OR destination, source

▪Example:

```
mov dl, 00111011b
or dl, 00001111b
```



OR		
x	y	$x \vee y$
0	0	0
0	1	1
1	0	1
1	1	1

47*XOR INSTRUCTION:

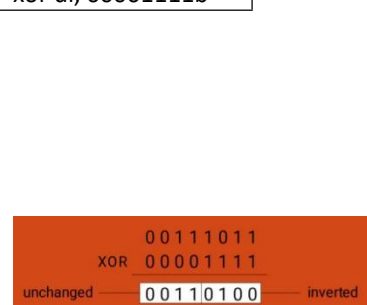
▪Performs a bitwise Boolean exclusive-OR operation between each pair of matching bits in two operands

▪Syntax:

XOR destination, source

▪Example:

```
mov dl, 00111011b
xor dl, 00001111b
```



XOR		
x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

48*CMP INSTRUCTION:

- Compares the destination operand to the source operand
- Nondestructive subtraction of source from destination (destination operand is not changed)
- Syntax: (OSZCAP)

CMP destination, source

unsigned	ZF	CF	signed	flags
destination<source	0	1	destination<source	SF != OF
destination>source	0	0	destination>source	SF == OF
destination=source	1	0	destination=source	ZF=1

Example: destination == source

```
mov al,5  
cmp al,5 ; Zero flag set
```

Example: destination < source

```
mov al,4  
cmp al,5 ; Carry flag set
```

Example: destination > source

```
mov al,6  
cmp al,5 ; ZF = 0, CF = 0
```

(both the Zero and Carry flags are clear)

49*SETTING AND CLEARING INDIVIDUAL FLAGS:

```
and al, 0 ; set Zero  
or al, 1 ; clear Zero  
or al, 80h ; set Sign  
and al, 7Fh ; clear Sign  
stc ; set Carry  
clc ; clear Carry  
mov al, 7Fh  
inc al ; set Overflow  
or eax, 0 ; clear Overflow
```

50*CONDITIONAL STRUCTURES:

- There are no high-level logic structures such as if-then-else, in the IA-32 instruction set. But, you can use combinations of comparisons and jumps to implement any logic structure.
- First, an operation such as CMP, AND or SUB is executed to modified the CPU flags.
Second, a conditional jump instruction tests the flags and changes the execution flow accordingly.

```
CMP AL, 0  
JZ L1  
:  
L1: JCOND INSTRUCTION
```

- A conditional jump instruction branches to a label when specific register or flag conditions are met
Jcond destination

- Four groups: (some are the same)

- 1.based on specific flag values
- 2.based on equality between operands
- 3.based on comparisons of unsigned operands
- 4.based on comparisons of signed operands

51*JUMPS BASED ON SPECIFIC FLAGS:

Mnemonic	Description	Flags
JZ	Jump if zero	ZF = 1
JNZ	Jump if not zero	ZF = 0
JC	Jump if carry	CF = 1
JNC	Jump if not carry	CF = 0
JO	Jump if overflow	OF = 1
JNO	Jump if not overflow	OF = 0
JS	Jump if signed	SF = 1
JNS	Jump if not signed	SF = 0
JP	Jump if parity (even)	PF = 1
JNP	Jump if not parity (odd)	PF = 0

52*JUMPS BASED ON EQUALITY:

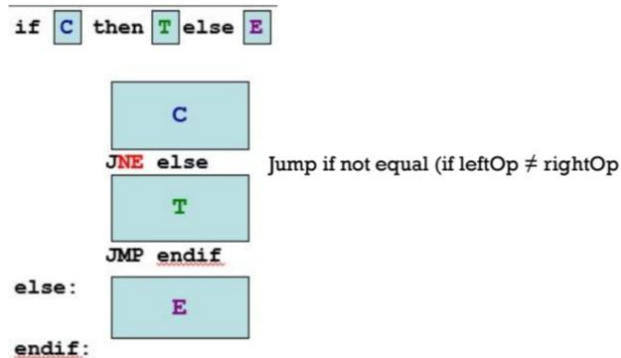
Mnemonic	Description
JE	Jump if equal (<i>leftOp</i> = <i>rightOp</i>)
JNE	Jump if not equal (<i>leftOp</i> ≠ <i>rightOp</i>)
JCXZ	Jump if CX = 0
JECXZ	Jump if ECX = 0

53*JUMPS BASED ON UNSIGNED COMPARISONS:

Mnemonic	Description
JA	Jump if above (if <i>leftOp</i> > <i>rightOp</i>)
JNBE	Jump if not below or equal (same as JA)
JAЕ	Jump if above or equal (if <i>leftOp</i> >= <i>rightOp</i>)
JNB	Jump if not below (same as JAЕ)
JB	Jump if below (if <i>leftOp</i> < <i>rightOp</i>)
JNAЕ	Jump if not above or equal (same as JB)
JBE	Jump if below or equal (if <i>leftOp</i> <= <i>rightOp</i>)
JNA	Jump if not above (same as JBE)
Mnemonic	Description
JG	Jump if greater (if <i>leftOp</i> > <i>rightOp</i>)
JNLE	Jump if not less than or equal (same as JG)
JGE	Jump if greater than or equal (if <i>leftOp</i> >= <i>rightOp</i>)
JNL	Jump if not less (same as JGE)
JL	Jump if less (if <i>leftOp</i> < <i>rightOp</i>)
JNGE	Jump if not greater than or equal (same as JL)
JLE	Jump if less than or equal (if <i>leftOp</i> <= <i>rightOp</i>)
JNG	Jump if not greater (same as JLE)

54*CONDITIONAL STRUCTURES:

IF STATEMENTS



BLOCK-STRUCTURED IF STATEMENTS

▪Assembly language programmers can easily translate logical statements written in C++/Java into assembly language. For example

```
if( op1 == op2 )
X = 1;
else
X = 2;

cmp ebx,ecx
ja next
mov eax,5
mov edx,6
next:
```

55*COMPOUND EXPRESSION WITH AND:

▪In the following example, if the first expression is false, the second expression is skipped:

```
if (al > bl) AND (bl > cl)
X = 1;
if (al > bl) AND (bl > cl)
X = 1;
    cmp al,bl ; first expression...
    ja L1
    jmp next
L1:
    cmp bl,cl ; second expression...
    ja L2
    jmp next
L2: ; both are true
    mov X,1 ; set X to 1
next:
```

▪But the following implementation uses 29% less code by reversing the first relational operator. We allow the program to "fall through" to the second expression:

```
if (al > bl) AND (bl > cl)
X = 1;
    cmp al,bl ; first expression...
    jbe next ; quit if false
    cmp bl,cl ; second expression...
    jbe next ; quit if false
    mov X,1 ; both are true
next:
```

56*COMPOUND EXPRESSION WITH OR:

▪In the following example, if the first expression is true, the second expression is skipped:

```
if (a! > b!) OR (b! > c!)  
X = 1;
```

▪We can use "fall-through" logic to keep the code as short as possible:

```
cmp al,bl ; is AL > BL?  
ja L1 ; yes  
cmp bl,cl ; no: is BL > CL?  
jbe next ; no: skip next statement  
L1:mov X,1 ; set X to 1  
next:
```

57*WHILE LOOPS:

▪A WHILE loop is really an IF statement followed by the body of the loop, followed by an unconditional jump to the top of the loop. Consider the following example:

```
while( eax < ebx)  
eax = eax + 1;
```

_while:

```
cmp eax,ebx ; check loop condition  
jae _endwhile ; false? exit loop  
inc eax ; body of loop  
jmp _while ; repeat the loop  
_endwhile:
```